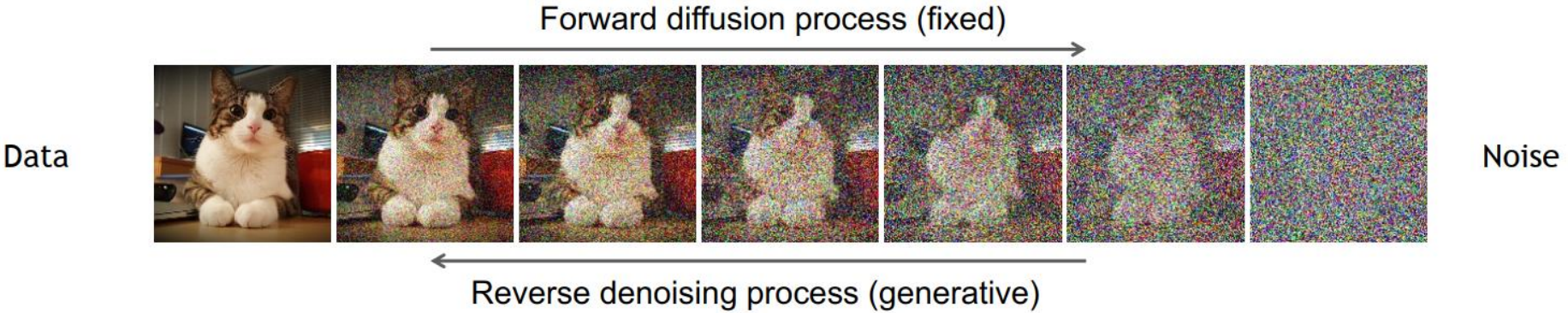
Diffusion Models

Denoising Diffusion Probabilistic Models

Learning to generate by denoising

Denoising diffusion models consist of two processes:

- Forward diffusion process that gradually adds noise to input
- Reverse denoising process that learns to generate data by denoising

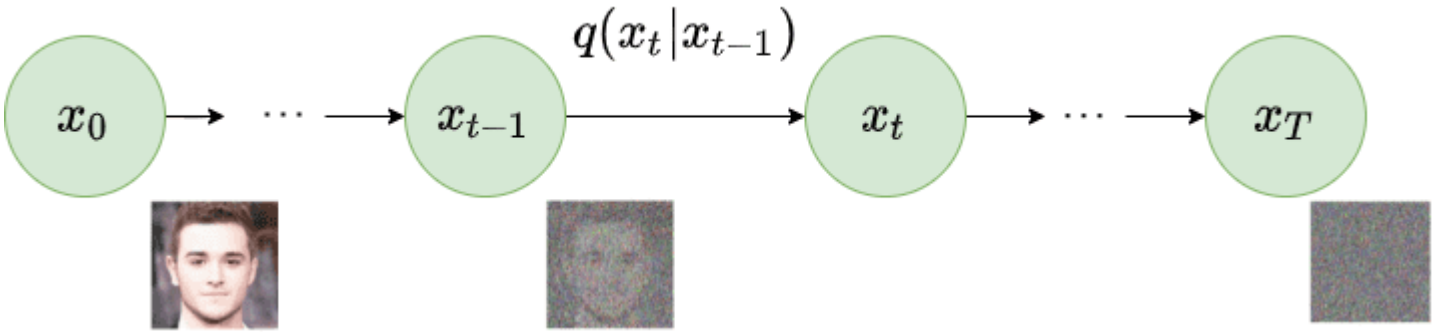


[Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015]
[Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020]
[Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021]

Diffusion Process

- Gradually add noise to input image $\boldsymbol{x_0}$ in a series of $\boldsymbol{T}$ time steps

- Neural network trained to recover original data



[Ho et al. '20] Denoising Diffusion Probabilistic Models

Forward Diffusion

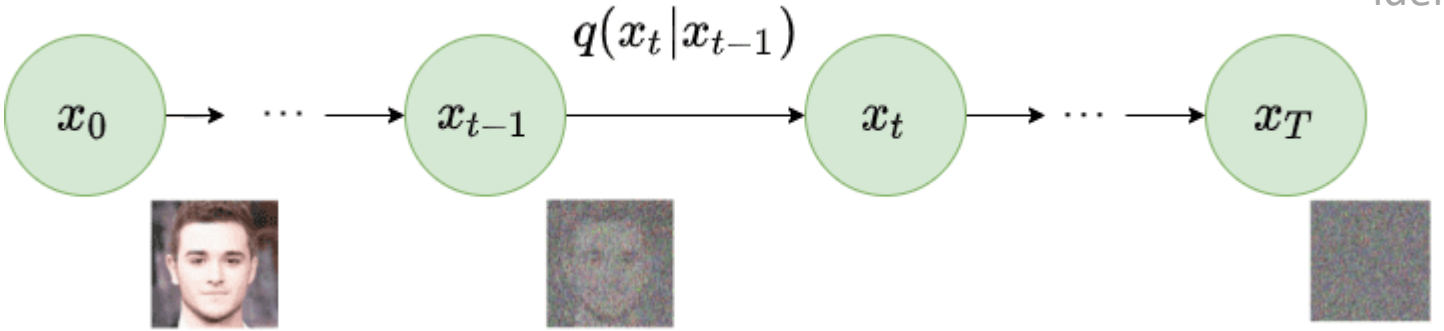
- Markov chain of T steps
  - Each step depends only on previous
- Adds noise to x_0 sampled from real distribution $q(x)$

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \boldsymbol{\mu}_t = \sqrt{1-\beta_t}x_{t-1}, \boldsymbol{\Sigma}_t = \beta_t \mathbf{I})$$

mean

variance

identity matrix

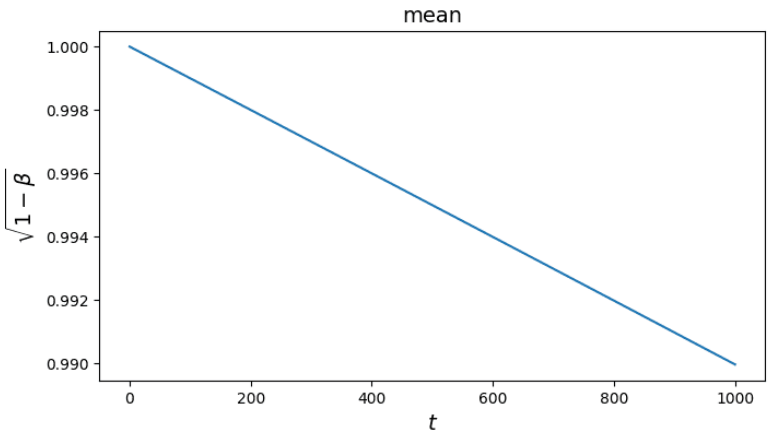


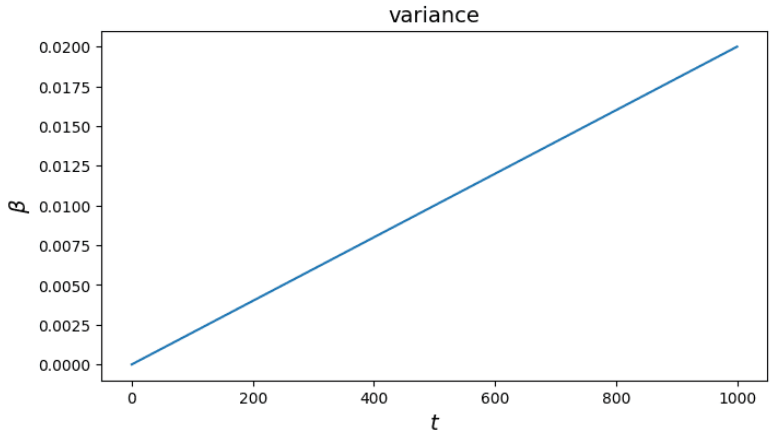
[Ho et al. '20] Denoising Diffusion Probabilistic Models

Forward Diffusion

$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \boldsymbol{\mu}_t = \sqrt{1-\beta_t}x_{t-1}, \boldsymbol{\Sigma}_t = \beta_t \mathbf{I})$$

Example: 1000 steps, linear beta schedule: 0.0001 to 0.02





https://erdem.pl/2023/11/step-by-step-visual-introduction-to-diffusion-models

Forward Diffusion

- Go from $\boldsymbol{x_0}$ to $\boldsymbol{x_T}$:

$$q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1})$$

- Efficiency?

Reparameterization

- Define $\alpha_t = 1 - \beta_t,\ \overline{\alpha_t} = \prod_{s=0}^{t} \alpha_s,\ \epsilon_0, \ldots, \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1}$$

$$\varepsilon_0, \ldots, \varepsilon_{t-2}, \varepsilon_{t-1} \sim \mathcal{N}(0, I)$$
$$\bar{\varepsilon}_0, \ldots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} \sim \mathcal{N}(0, I)$$
$$\varepsilon \sim \mathcal{N}(0, I)$$

Reparameterization

- Define $\alpha_t = 1 - \beta_t,\ \overline{\alpha_t} = \prod_{s=0}^{t} \alpha_s,\ \epsilon_0, \ldots, \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1}$$
$$= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1}$$

$$\varepsilon_0, \ldots, \varepsilon_{t-2}, \varepsilon_{t-1} \sim \mathcal{N}(0, I)$$
$$\bar{\varepsilon}_0, \ldots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} \sim \mathcal{N}(0, I)$$
$$\varepsilon \sim \mathcal{N}(0, I)$$

Reparameterization

- Define $\alpha_t = 1 - \beta_t,\ \overline{\alpha_t} = \prod_{s=0}^{t} \alpha_s,\ \epsilon_0, \ldots, \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$$
\begin{aligned}
x_t &= \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \varepsilon_{t-2} \right) + \sqrt{1-\alpha_t}\, \varepsilon_{t-1}
\end{aligned}
$$

$$
\begin{aligned}
\varepsilon_0, \ldots, \varepsilon_{t-2}, \varepsilon_{t-1} &\sim \mathcal{N}(0, I) \\
\bar{\varepsilon}_0, \ldots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} &\sim \mathcal{N}(0, I) \\
\varepsilon &\sim \mathcal{N}(0, I)
\end{aligned}
$$

Reparameterization

- Define $\alpha_t = 1 - \beta_t,\ \overline{\alpha_t} = \prod_{s=0}^{t} \alpha_s,\ \epsilon_0, \ldots, \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$$
\begin{aligned}
x_t &= \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \varepsilon_{t-2} \right) + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t\, \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\, \varepsilon_{t-2} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1}
\end{aligned}
$$

$$
\begin{aligned}
&\varepsilon_0, \ldots, \varepsilon_{t-2}, \varepsilon_{t-1} \sim \mathcal{N}(0, I) \\
&\bar{\varepsilon}_0, \ldots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} \sim \mathcal{N}(0, I) \\
&\varepsilon \sim \mathcal{N}(0, I)
\end{aligned}
$$

Reparameterization

- Define $\alpha_t = 1 - \beta_t,\ \overline{\alpha_t} = \prod_{s=0}^{t} \alpha_s,\ \epsilon_0, \ldots, \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$$
\begin{aligned}
x_t &= \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \varepsilon_{t-2} \right) + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t\, \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\, \varepsilon_{t-2} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t\, \alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_t\, \alpha_{t-1}}\, \bar{\varepsilon}_{t-2}
\end{aligned}
$$

$$
\begin{aligned}
\varepsilon_0, \ldots, \varepsilon_{t-2}, \varepsilon_{t-1} &\sim \mathcal{N}(0, I) \\
\bar{\varepsilon}_0, \ldots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} &\sim \mathcal{N}(0, I) \\
\varepsilon &\sim \mathcal{N}(0, I)
\end{aligned}
$$

Explained on later slide

Reparameterization

- Define $\alpha_t = 1 - \beta_t,\ \overline{\alpha_t} = \prod_{s=0}^{t} \alpha_s,\ \epsilon_0, \ldots, \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$$
\begin{aligned}
x_t &= \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \varepsilon_{t-2} \right) + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t\, \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\, \varepsilon_{t-2} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t\, \alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_t\, \alpha_{t-1}}\, \bar{\varepsilon}_{t-2} \\
&\vdots \\
&= \sqrt{\alpha_t\, \alpha_{t-1} \cdots \alpha_1}\, x_0 + \sqrt{1-\alpha_t\, \alpha_{t-1} \cdots \alpha_1}\, \varepsilon
\end{aligned}
$$

$\varepsilon_0, \ldots, \varepsilon_{t-2}, \varepsilon_{t-1} \sim \mathcal{N}(0, I)$

$\bar{\varepsilon}_0, \ldots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} \sim \mathcal{N}(0, I)$

$\varepsilon \sim \mathcal{N}(0, I)$

Explained on later slide

Reparameterization

- Define $\alpha_t = 1 - \beta_t,\ \overline{\alpha_t} = \prod_{s=0}^{t} \alpha_s,\ \epsilon_0, \ldots, \epsilon_{t-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

$$
\begin{aligned}
x_t &= \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t}\, x_{t-1} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_{t-1}}\, \varepsilon_{t-2} \right) + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t\, \alpha_{t-1}}\, x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\, \varepsilon_{t-2} + \sqrt{1-\alpha_t}\, \varepsilon_{t-1} \\
&= \sqrt{\alpha_t\, \alpha_{t-1}}\, x_{t-2} + \sqrt{1-\alpha_t\, \alpha_{t-1}}\, \bar{\varepsilon}_{t-2} \\
&\vdots \\
&= \sqrt{\alpha_t\, \alpha_{t-1} \cdots \alpha_1}\, x_0 + \sqrt{1-\alpha_t\, \alpha_{t-1} \cdots \alpha_1}\, \varepsilon \\
&= \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon
\end{aligned}
$$

$\varepsilon_0, \ldots, \varepsilon_{t-2}, \varepsilon_{t-1} \sim \mathcal{N}(0, I)$
$\bar{\varepsilon}_0, \ldots, \bar{\varepsilon}_{t-2}, \bar{\varepsilon}_{t-1} \sim \mathcal{N}(0, I)$
$\varepsilon \sim \mathcal{N}(0, I)$

Explained on later slide

Reparameterization – Step line 4 to 5

$$= \sqrt{\alpha_t\,\alpha_{t-1}}\,x_{t-2} + \sqrt{\alpha_t(1-\alpha_{t-1})}\,\varepsilon_{t-2} + \sqrt{1-\alpha_t}\,\varepsilon_{t-1}$$

$X + Y$

Reparameterization Trick	Underlying Normal Distribution
$0 + \sqrt{\alpha_t(1-\alpha_{t-1})}\,\varepsilon_{t-2}$	$X \sim \mathcal{N}(0,\ \alpha_t(1-\alpha_{t-1})\,I)$
$0 + \sqrt{1-\alpha_t}\,\varepsilon_{t-1}$	$Y \sim \mathcal{N}(0,\ (1-\alpha_t)\,I)$

Recall:

If $X \sim \mathcal{N}(\mu_X, \sigma_X^2)$ $\quad Y \sim \mathcal{N}(\mu_Y, \sigma_Y^2)$

$Z = X + Y$

Then $Z \sim \mathcal{N}(\mu_X + \mu_Y,\ \sigma_X^2 + \sigma_Y^2)$

$Z \sim \mathcal{N}(0,\ \sigma_X^2 + \sigma_Y^2)$

$Z \sim \mathcal{N}(0,\ (1-\alpha_t\alpha_{t-1})\,I)$

$$\sigma_X^2 + \sigma_Y^2 = \alpha_t(1-\alpha_{t-1}) + (1-\alpha_t)$$
$$= \cancel{\alpha_t} - \alpha_t\alpha_{t-1} + 1 - \cancel{\alpha_t}$$
$$= 1 - \alpha_t\alpha_{t-1}$$

Reparameterization Trick

$$0 + \sqrt{1-\alpha_t\alpha_{t-1}}\,\bar{\varepsilon}_{t-2}$$

$$= \sqrt{\alpha_t\,\alpha_{t-1}}\,x_{t-2} + \sqrt{1-\alpha_t\,\alpha_{t-1}}\,\bar{\varepsilon}_{t-2}$$

https://roysubhradip.hashnode.dev/a-beginners-guide-to-diffusion-models-understanding-the-basics-and-beyond

Reverse Diffusion

- $x_{T\to\infty}$ becomes a Gaussian distribution

- Reverse distribution $q(x_{t-1}|x_t)$
  - Sample $x_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and run reverse process
  - Generates a novel data point from original distribution

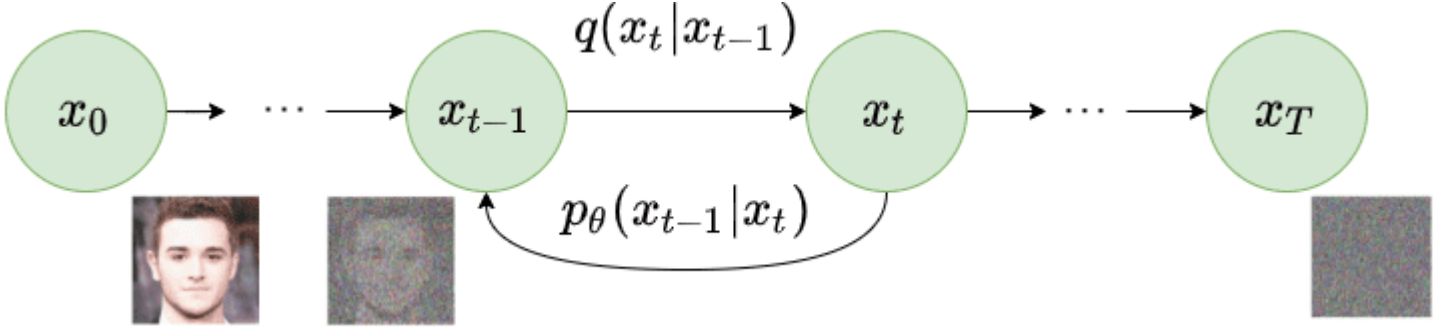
- How to model reverse process?

Approximate Reverse Process

- Approximate $q(x_{t-1}|x_t)$ with parameterized model p_θ (e.g., deep network)

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

$$p_\theta(x_{0:T}) = p_\theta(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)$$



[Ho et al. '20] Denoising Diffusion Probabilistic Models

Training a Diffusion Model

- Optimize negative log-likelihood of training data

$L = -\log(p_\theta(x_0))$ ← intractable depends on $x_1, x_2 \dots x_T$

→ Instead, optimize Variational Lower Bound:

$$L_{VLB} = \mathbb{E}_q[D_{KL}(q(x_T|x_0)||p_\theta(x_T))]_{L_T} + \sum_{t=2}^{T} D_{KL}(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)) - \log p_\theta(x_0|x_1)]$$

L_T: No learnable parameters; Gaussian noise → constant → ignore

L_{t-1}: Stepwise denoising term

L_0: Reconstruction term of last denoising step → ignore

Training a Diffusion Model

- $L_{t-1} = D_{KL}\left(q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)\right)$
- Comparing two Gaussian distributions, closed form computation

- $L_{t-1} \propto \|\tilde{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\|^2$
- Predicts diffusion posterior mean

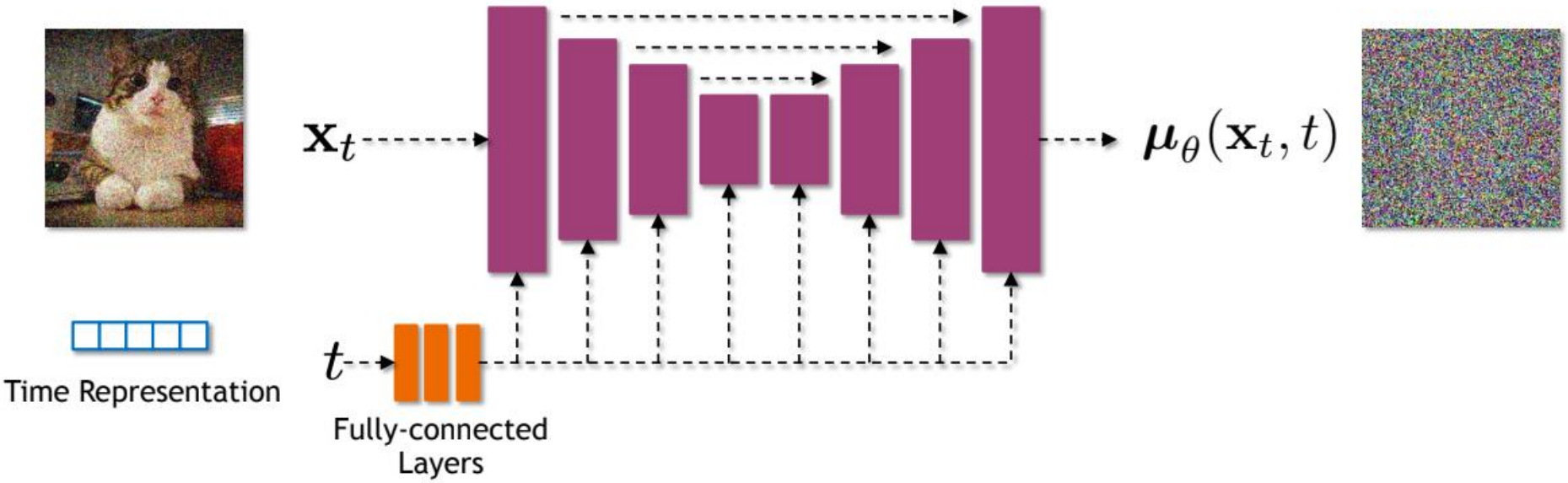
Nice derivations: https://lilianweng.github.io/posts/2021-07-11-diffusion-models

Diffusion Model Architecture

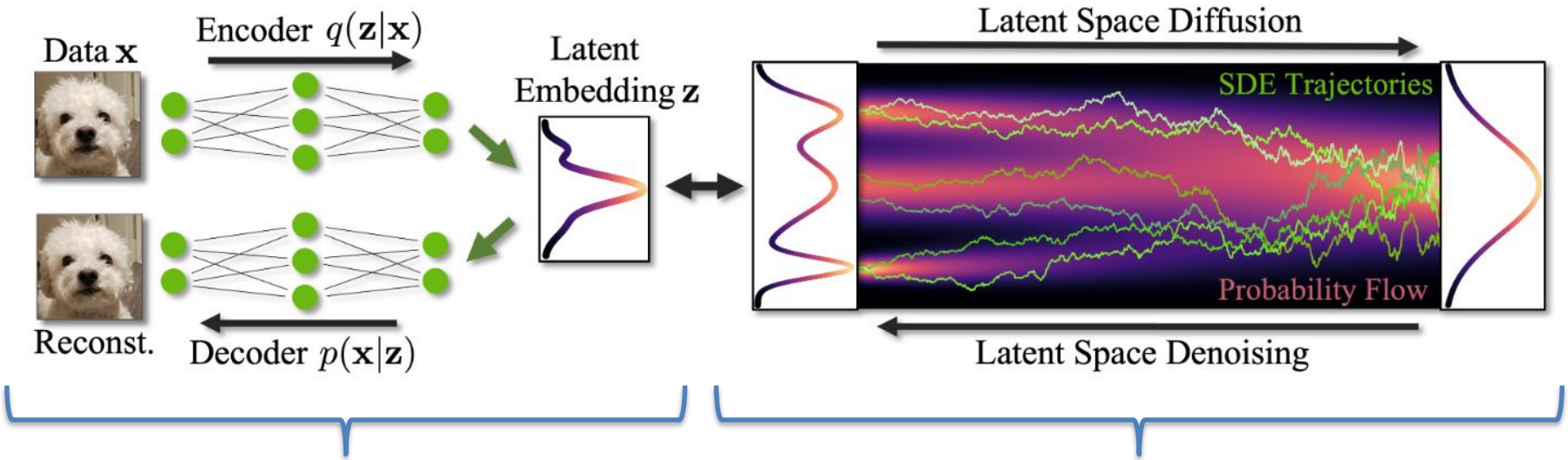
- Input and output dimensions must match
- Highly flexible to architecture design
- Commonly implemented with U-Net architecture

Diffusion Model Architecture

Example U-Net as diffusion model



Latent Diffusion



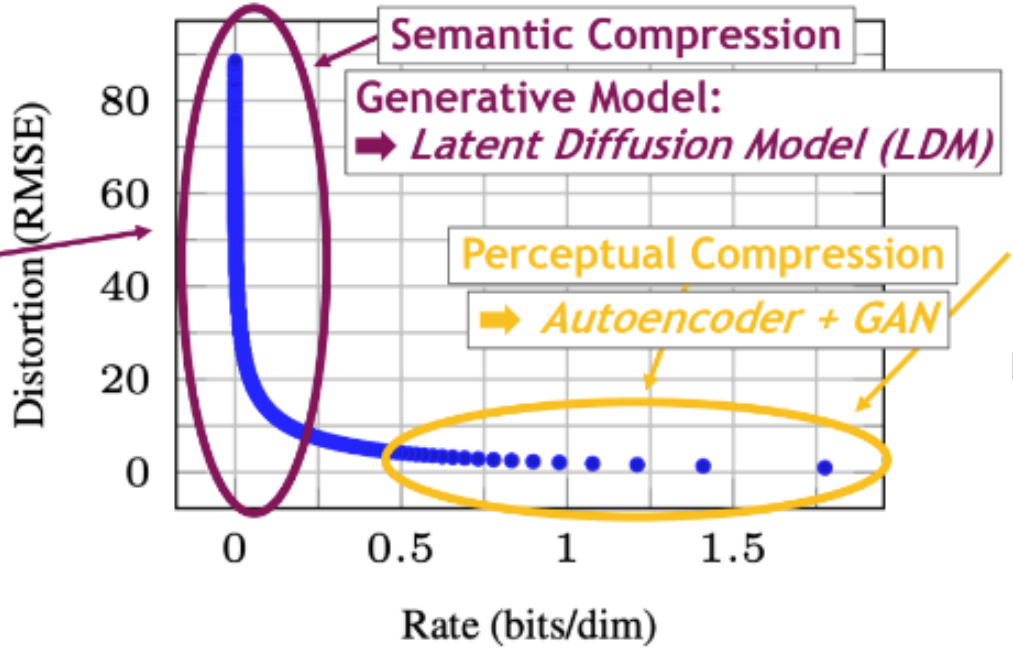
Stage 1: Train AE / VAE / VQ-VAE / VQ-GAN

Stage 2: Diffusion in Latent Space

https://neurips2023-ldm-tutorial.github.io/

Latent Diffusion

Large-scale
Image Structure

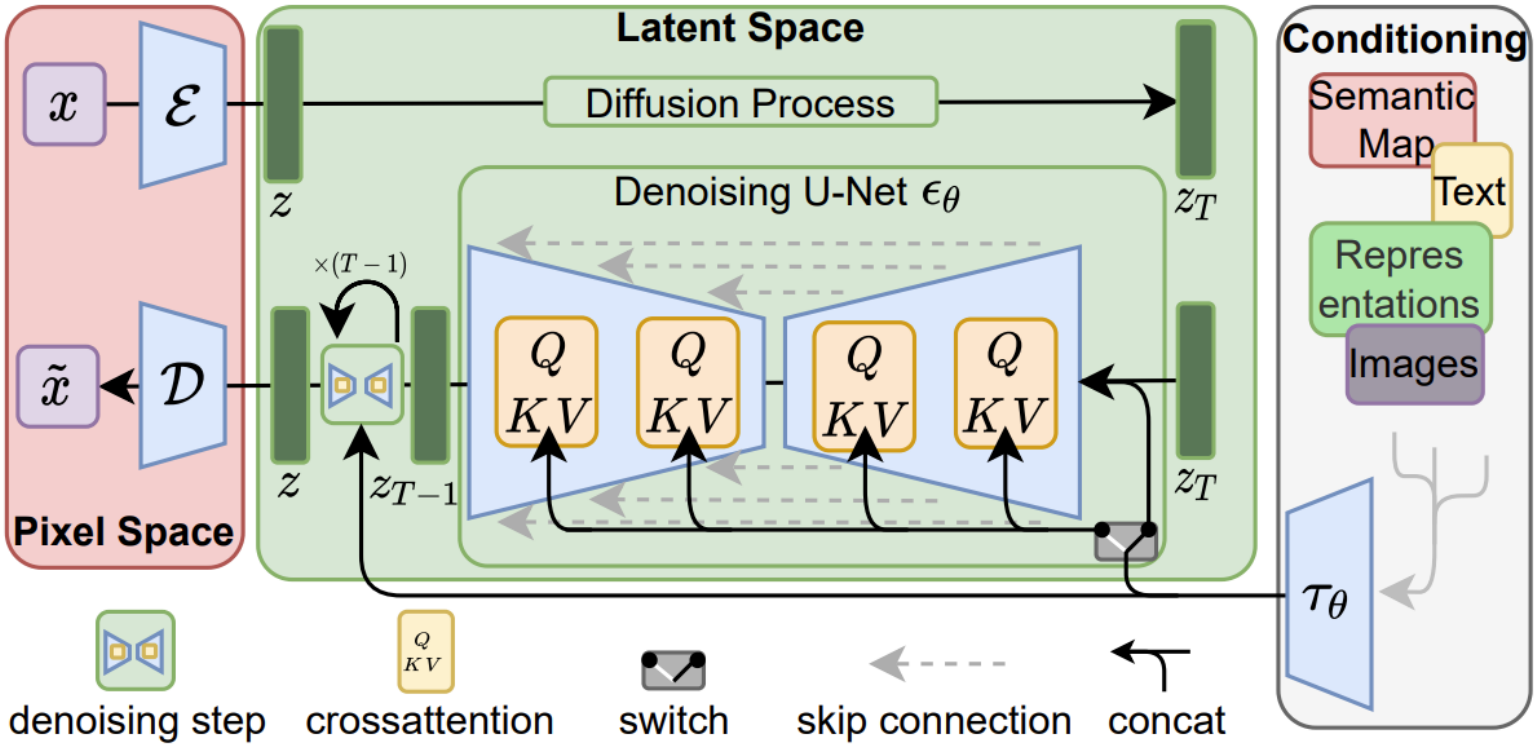


Local, Imperceptible
Details

LDMs: Latent diffusion model for large-scale structure, Autoencoder/GAN for local details.

https://neurips2023-ldm-tutorial.github.io/

Latent Diffusion

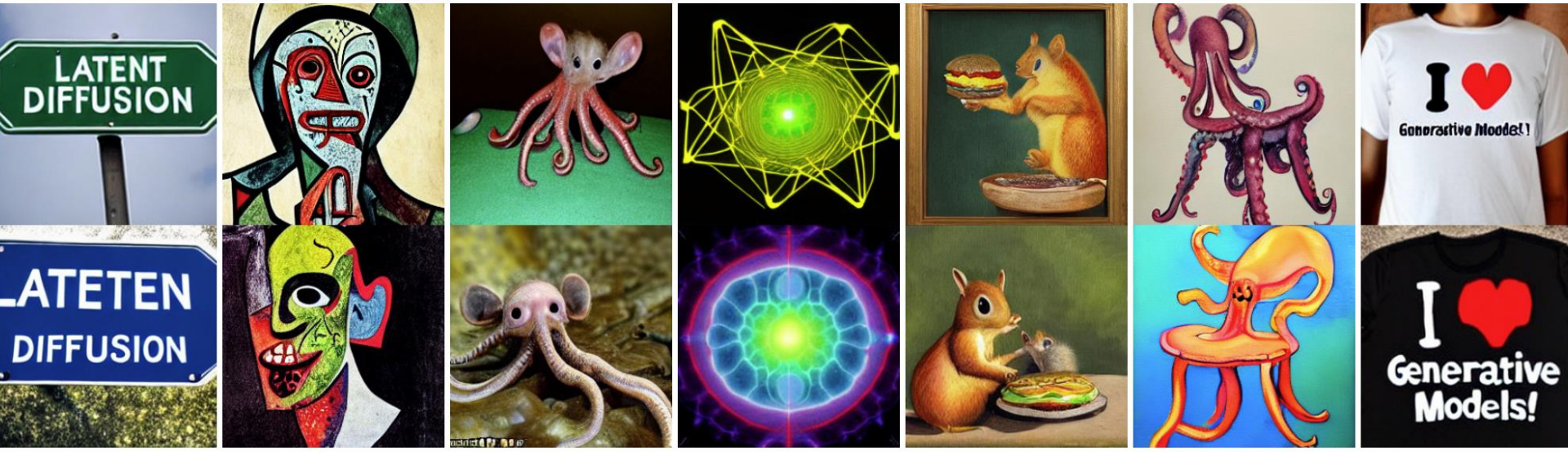


[Rombach et al. 21] Latent Diffusion

Latent Diffusion

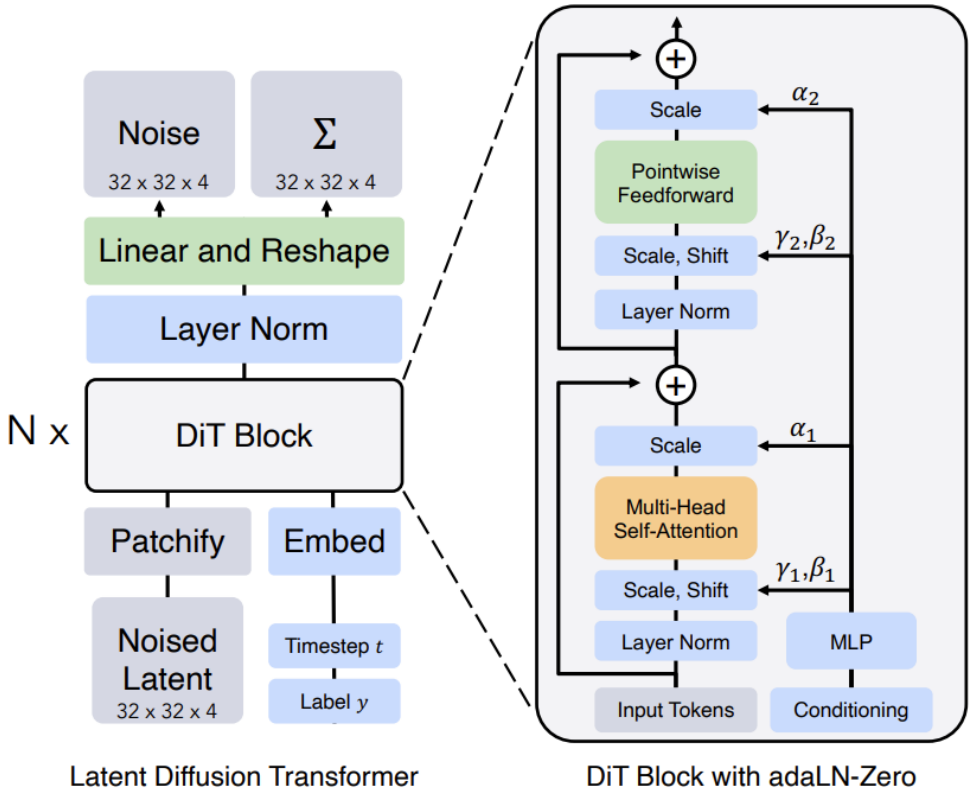
Text-to-Image Synthesis on LAION. 1.45B Model.

'A street sign that reads "Latent Diffusion" '	'A zombie in the style of Picasso'	'An image of an animal half mouse half octopus'	'An illustration of a slightly conscious neural network'	'A painting of a squirrel eating a burger'	'A watercolor painting of a chair that looks like an octopus'	'A shirt with the inscription: "I love generative models!" '



[Rombach et al. 21] Latent Diffusion

Diffusion Transformers (DiT)



[Peebles and Xie 23] Diffusion Transformers

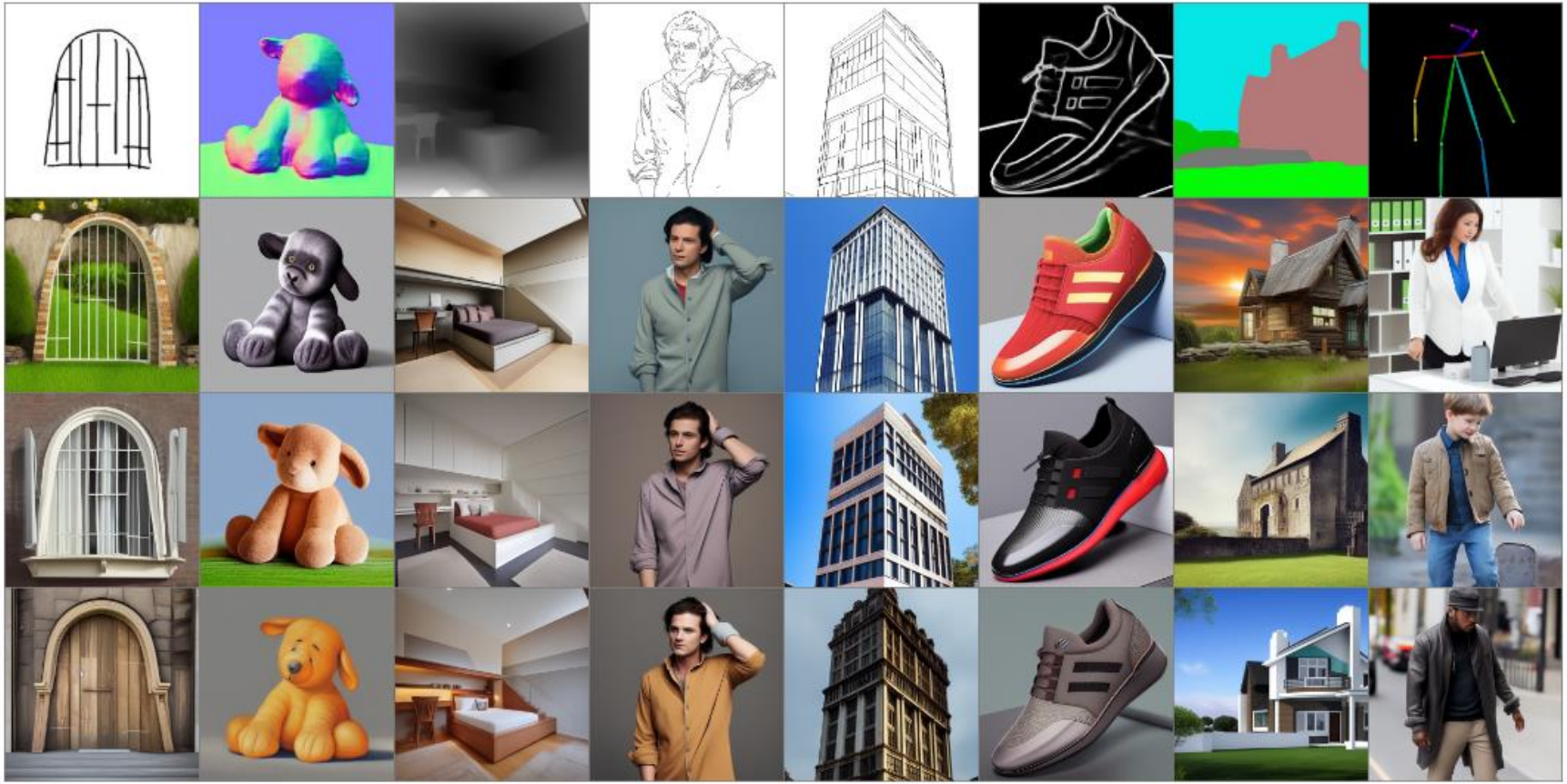
Diffusion Transformers (DiT)

[Peebles and Xie 23] Diffusion Transformers

ControlNet: Conditional Control for Text-to-Image Diffusion Models

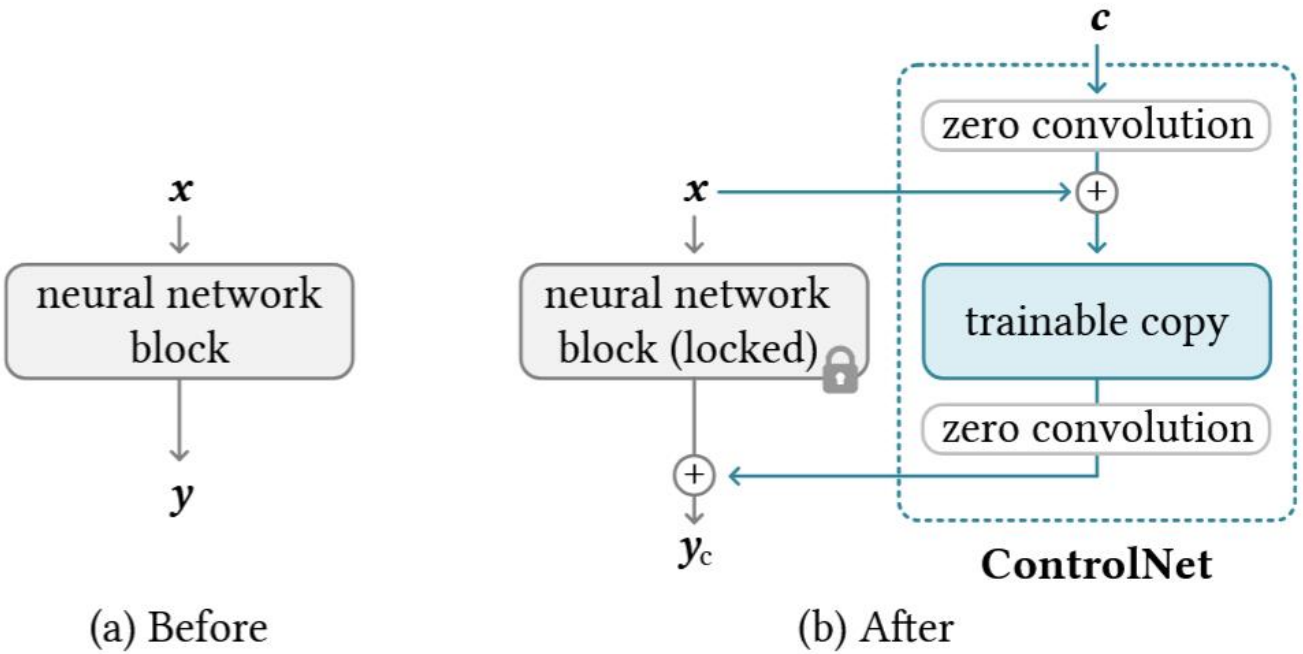
Condition

Generations



ControlNet: Conditional Control for Text-to-Image Diffusion Models

- Learn to control a pretrained model with condition c



[Zhang et al. '23] Adding Conditional Control to Text-to-Image Diffusion Models

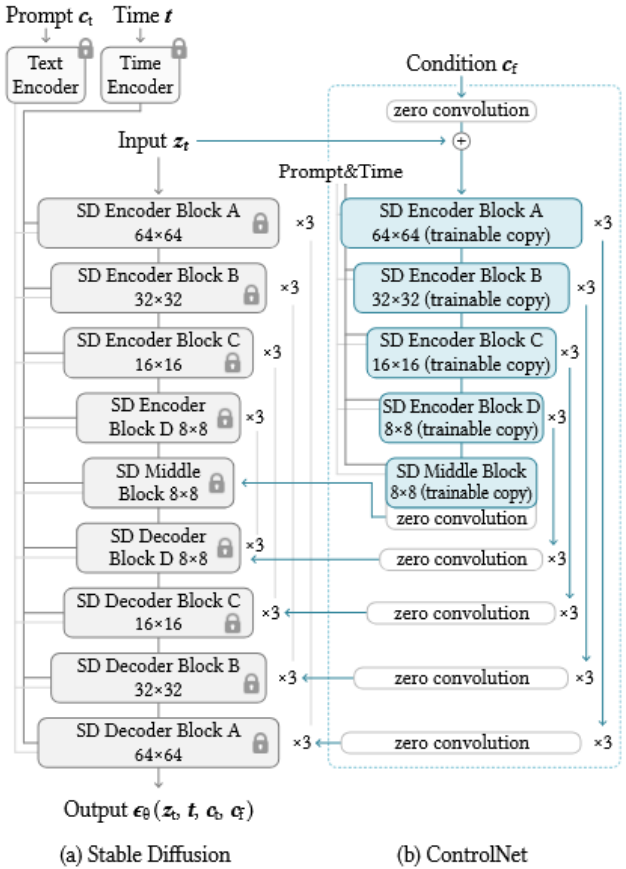
ControlNet: Conditional Control for Text-to-Image Diffusion Models

- Use weights of pretrained network in a locked copy (preserves model) & a trainable copy (learns condition)

→ Allows training with small data

- Important: Initialization with zero convolutions

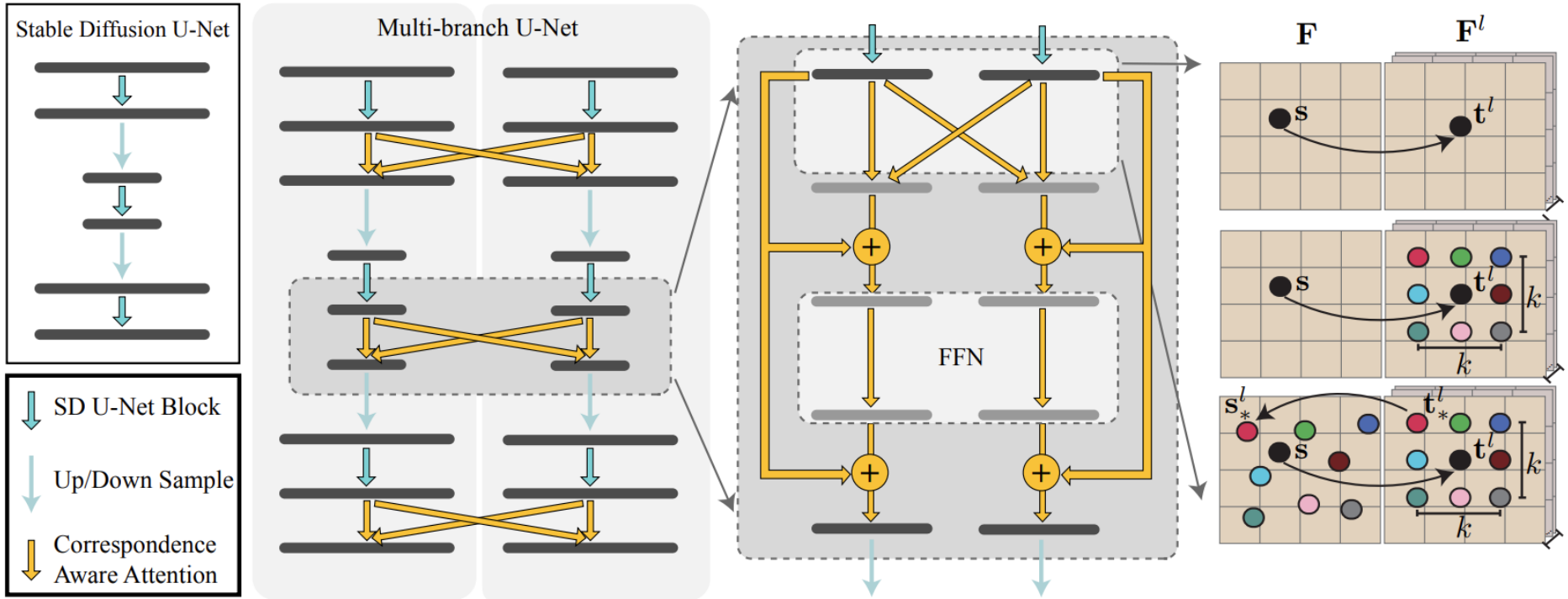
→ No initial distortion from ControlNet before training starts



[Zhang et al. '23] Adding Conditional Control to Text-to-Image Diffusion Models

Multi-view Diffusion Models

MVDiffusion



[Tang et al. 23] Tang et al.

MVDiffusion

"This kitchen is a charming blend of rustic and modern, featuring a large reclaimed wood island with marble countertop, a sink surrounded by cabinets. A stainless-steel refrigerator stands tall. To the right of the sink, built-in wooden cabinets painted in a muted."

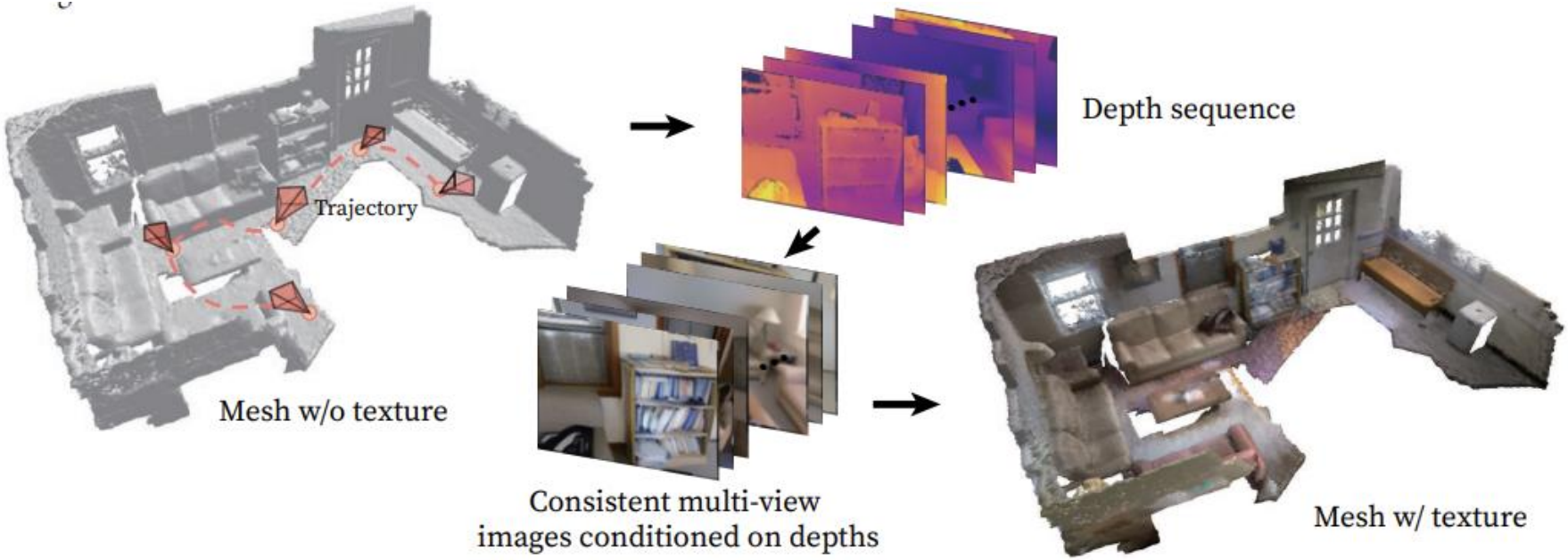
Consistent Multi-view Images

Closed-Loop Panoramic Image

"A living room with multiple couches and a coffee table. A wooden book shelf filled with lots of books next to a door. A white refrigerator sitting next to a wooden bench."

[Tang et al. 23] Tang et al.

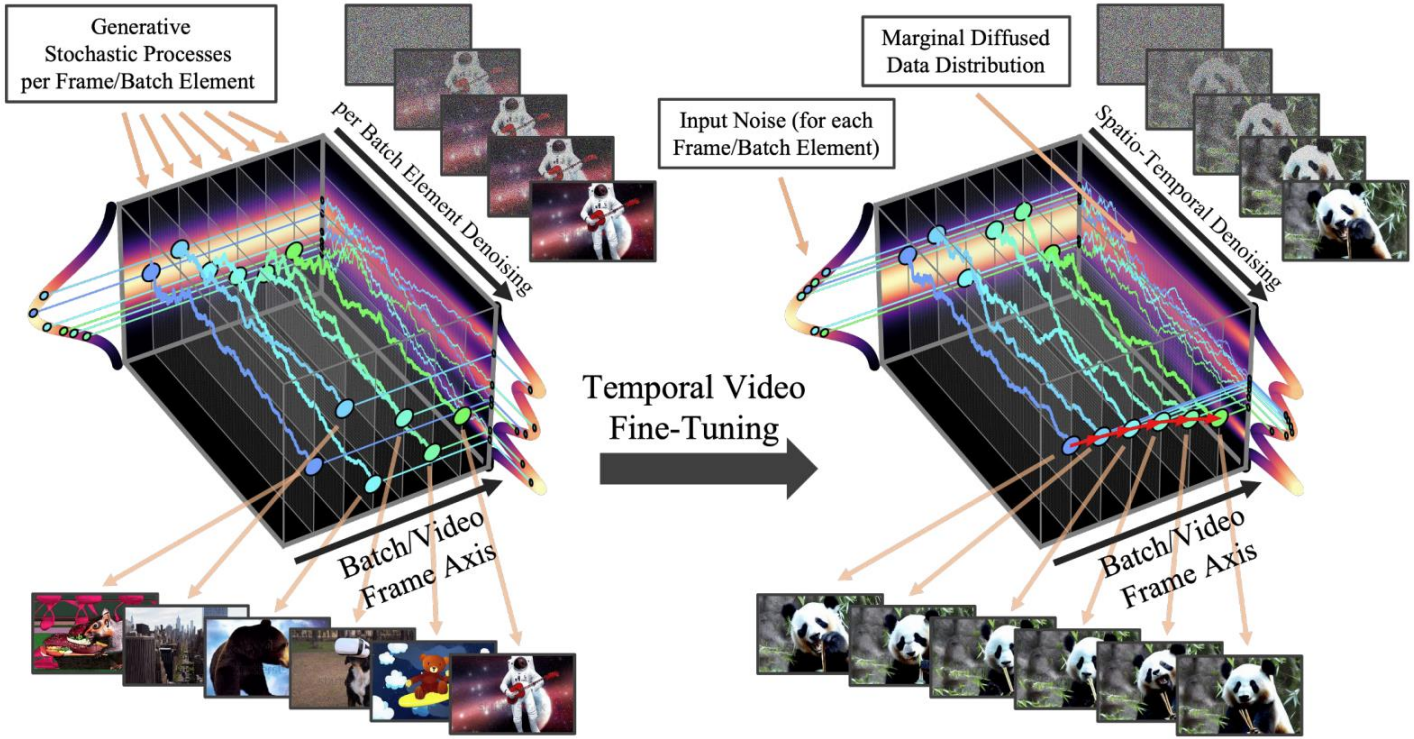
MVDiffusion



[Tang et al. 23] Tang et al.

Video Diffusion Models

Align your Latents

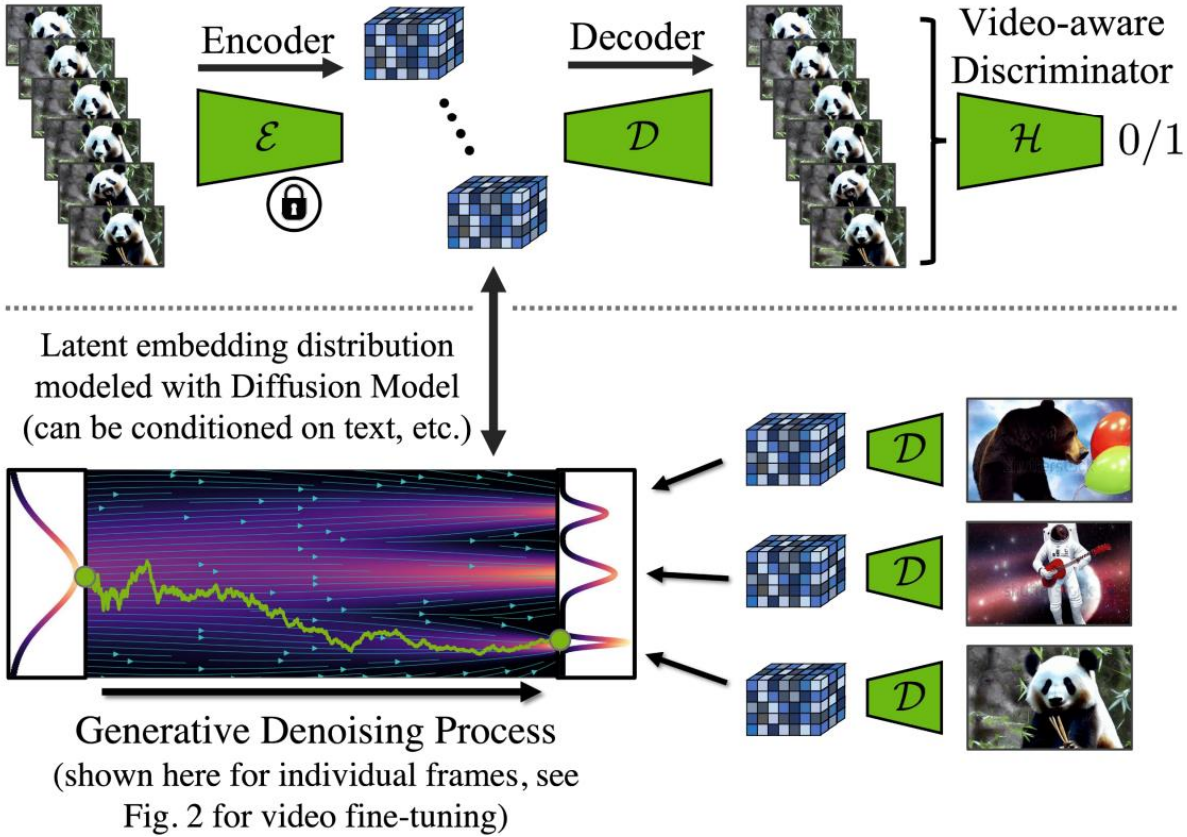


Before temporal video fine-tuning, different batch samples are independent.

After temporal video fine-tuning, samples are aligned to form a video sequence (after applying the LDM decoder).

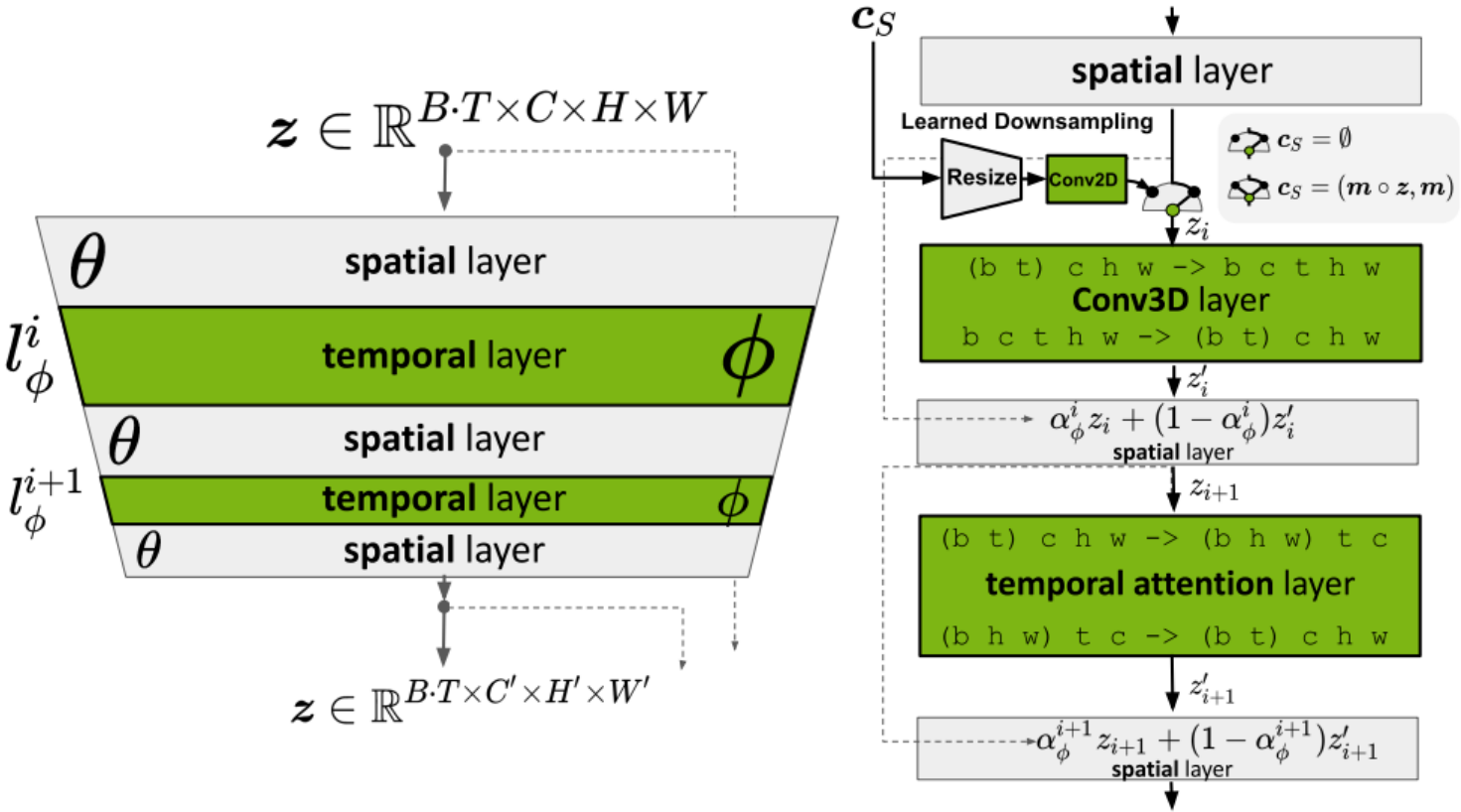
[Blattmann et al. 22] Latent Align your Latents

Align your Latents



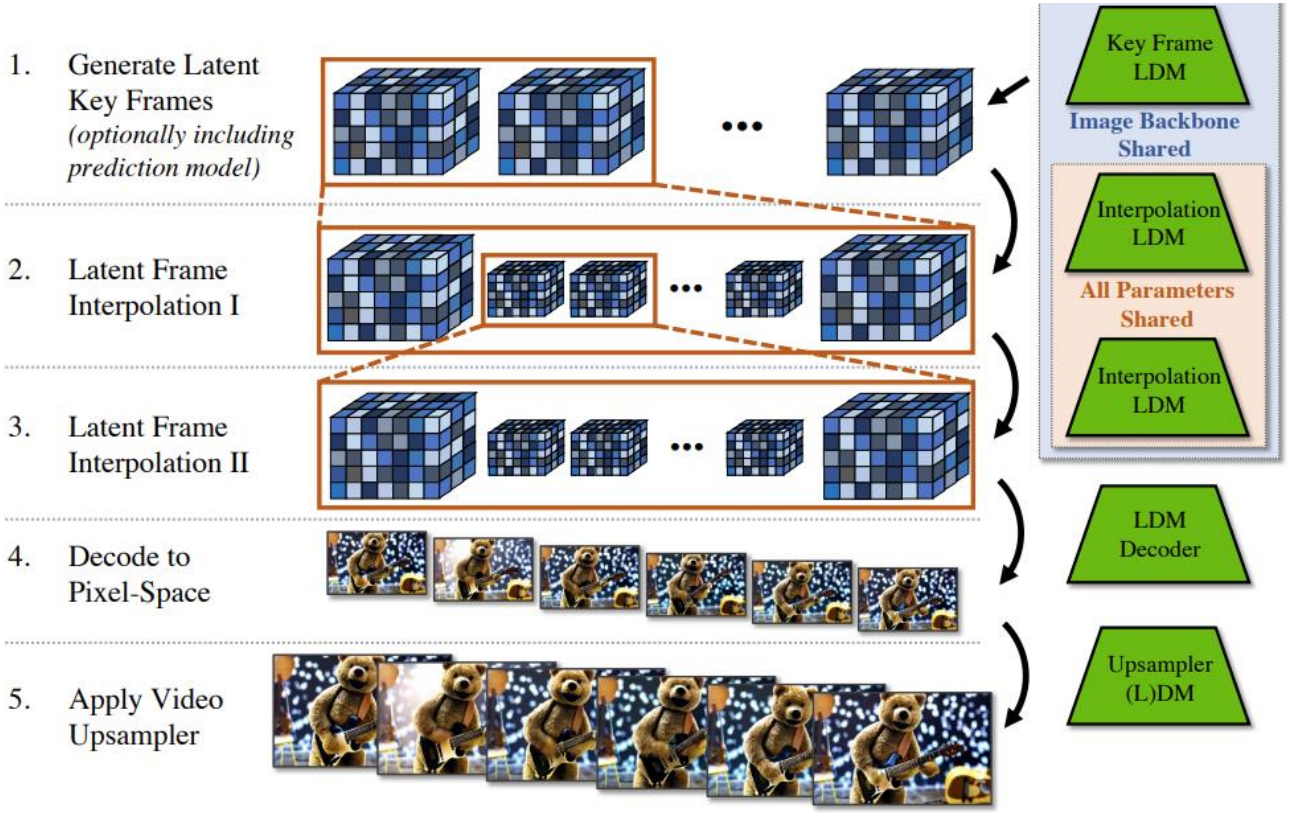
[Blattmann et al. 22] Latent Align your Latents

Align your Latents



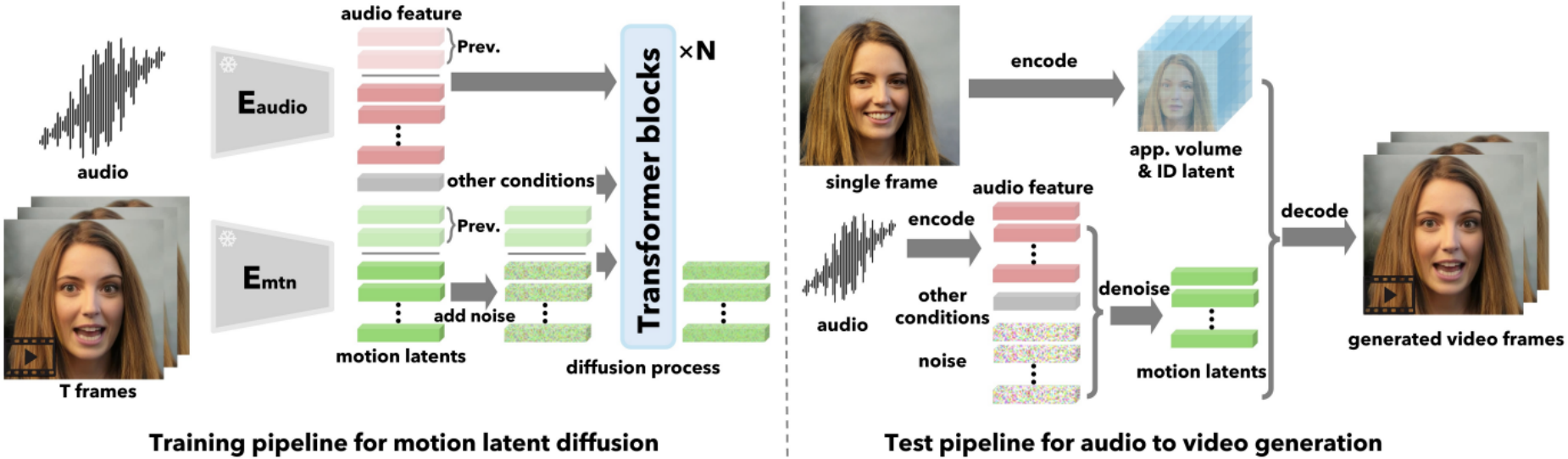
[Blattmann et al. 22] Latent Align your Latents

Align your Latents



[Blattmann et al. 22] Latent Align your Latents

VASA-1



[Xu et al. 24] VASA-1

VASA-1

[Xu et al. 24] VASA-1

VASA-1

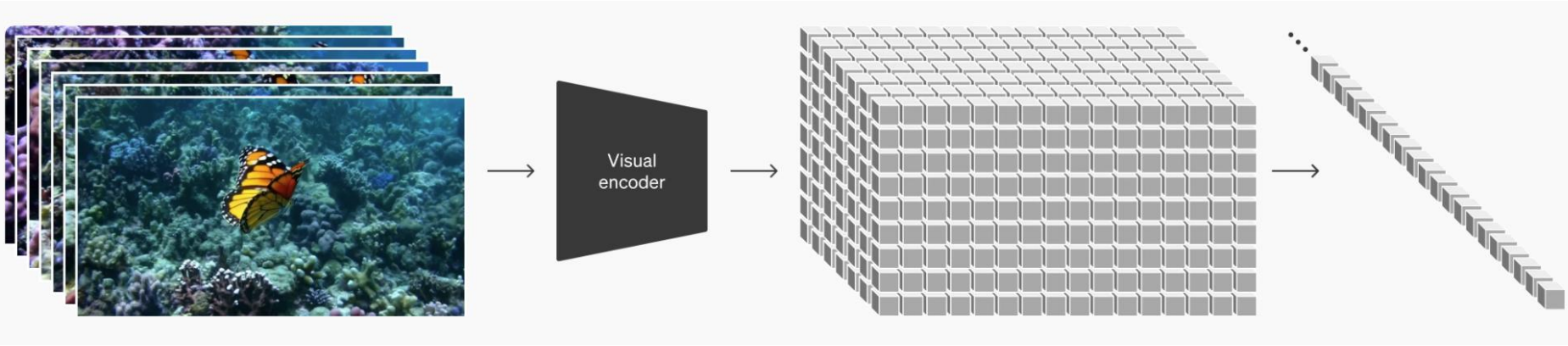
[Xu et al. 24] VASA-1

Sora

Sora

Sora

- Temporal Diffusion Transformer
- VAE vs VQ-GAN vs … ?
- Temporal window?
- Pre-trained on any images?



“Video Compressor”

Luma Dream Machine

Jiaming Song's Talk https://youtube.com/live/oilWwsXZamA

Latent Design

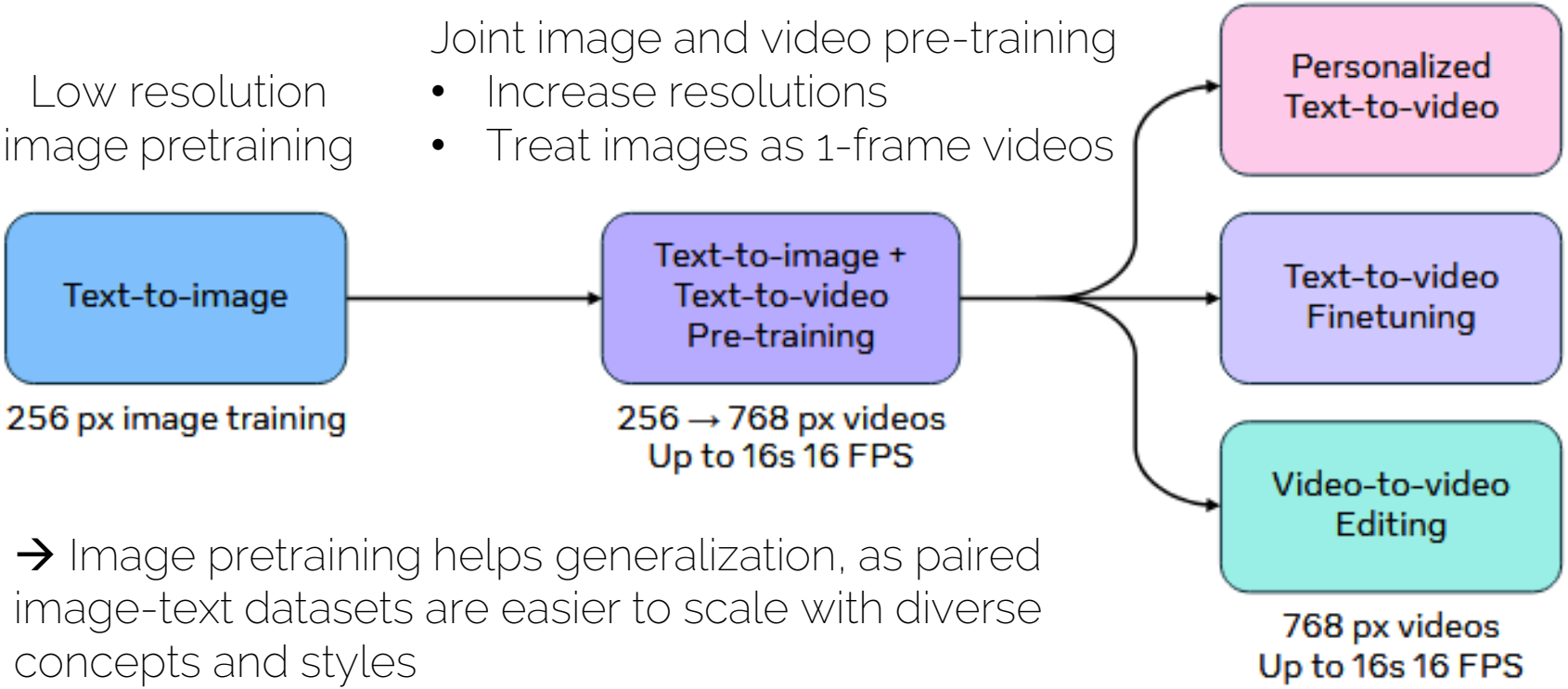
- Pre-trained image-based LDM
  - Can leverage massive amounts of pre-training
  - Issues in low-level temporal stability

- Training directly on videos
  - Can't use pre-trained image models
  - Potentially better low-level stability

Movie Gen: A Cast of Media Foundation Models

Low resolution image pretraining

Joint image and video pre-training
- Increase resolutions
- Treat images as 1-frame videos

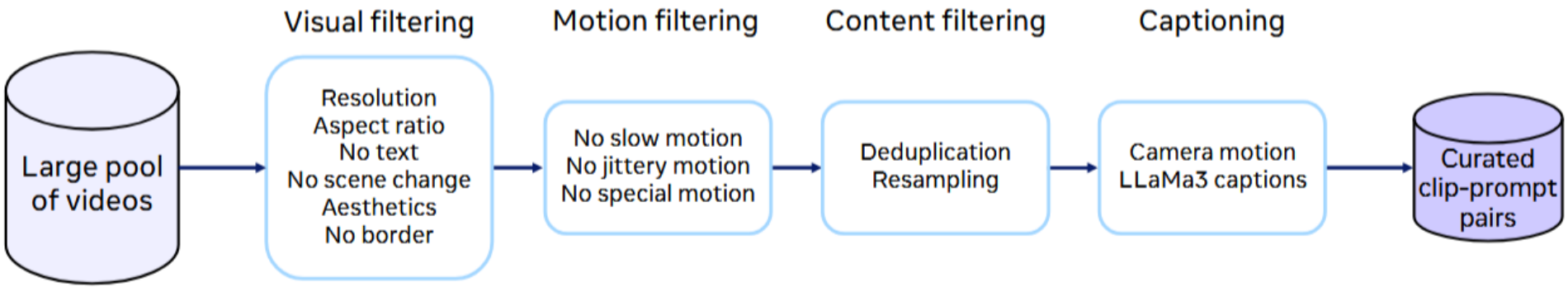


→ Image pretraining helps generalization, as paired image-text datasets are easier to scale with diverse concepts and styles

[Polyak et al. '25] Movie Gen

Movie Gen: A Cast of Media Foundation Models

Video preprocessing pipeline



[Polyak et al. '25] Movie Gen

Movie Gen: A Cast of Media Foundation Models

Joint image-video generation model

Spatio-temporally compressed latent space

Length 4-16s

Different aspect ratios (1:1, 9:16, 16:9, ..)

768x768 resolution (scaled based on aspect ratio)

TAE = Temporal Autoencoder

TAE Encoder

Patchify

Timestep t

Scale & Shift

Nx Transformer Blocks

TAE Decoder

Cross-attention

+pos embed

UL2

Long-prompt MetaCLIP

ByT5

An emu holding a sign says "No, Movie Gen is the best"

+ Spatial upsampler in image space

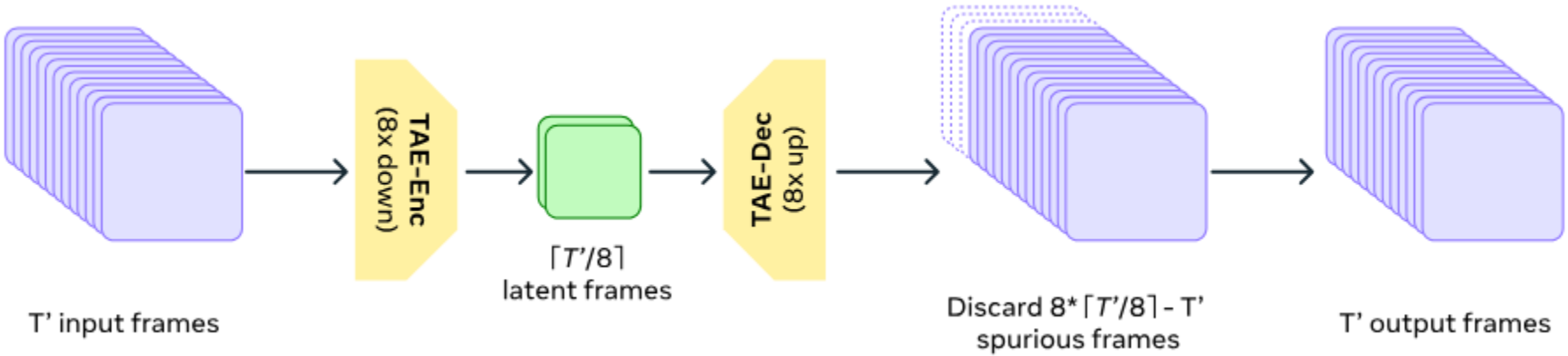
Gaussian noise

Flow matching training objective

[Polyak et al. '25] Movie Gen

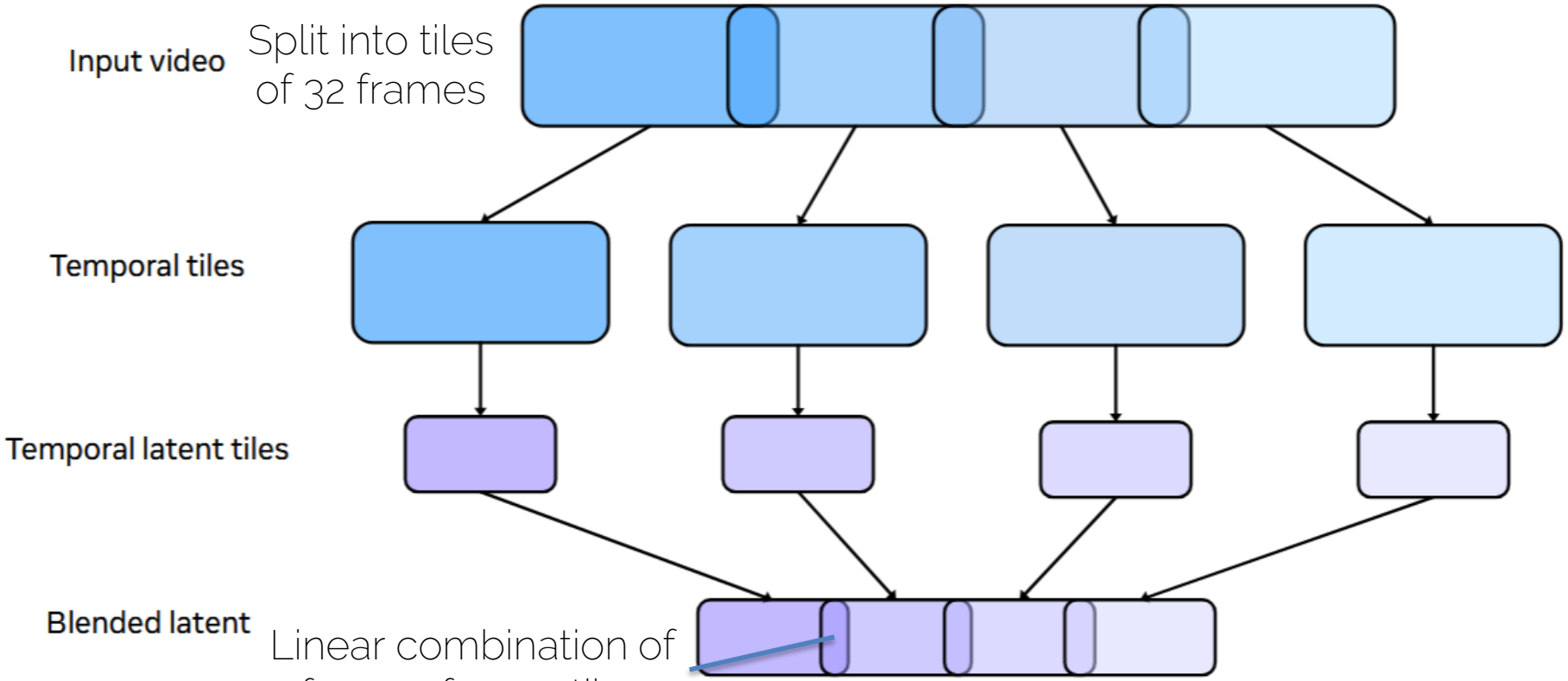
Movie Gen: A Cast of Media Foundation Models

Spatial and temporal (i.e., fewer latent frames) compression in the latent space



[Polyak et al. '25] Movie Gen

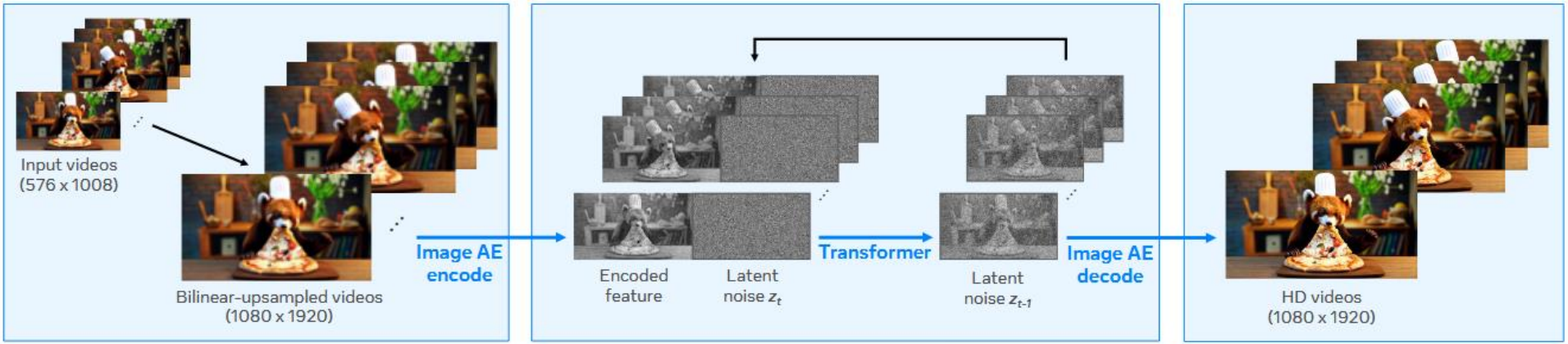
Movie Gen: A Cast of Media Foundation Models



[Polyak et al. '25] Movie Gen

Movie Gen: A Cast of Media Foundation Models

Spatial Upsampler converts 768 px videos to full HD (1080p) resolution



[Polyak et al. '25] Movie Gen

Text-to-Video

A girl is running across a beach and holding a kite. She's wearing jean shorts and a yellow t-shirt. The sun is shining down.

[Polyak et al. '25] Movie Gen

Personalization

A man is doing a scientific experiment in a lab with rainbow wallpaper. The man has a serious expression and is wearing glasses. He is wearing a white lab coat with a pen in the pocket. The man pours liquid into a glass beaker and a cloud of white smoke blooms.

[Polyak et al. '25] Movie Gen

Text-based Editing

[Polyak et al. '25] Movie Gen

Audio Generation

Wheels spinning, and a slamming sound as the skateboard lands on concrete.

[Polyak et al. '25] Movie Gen

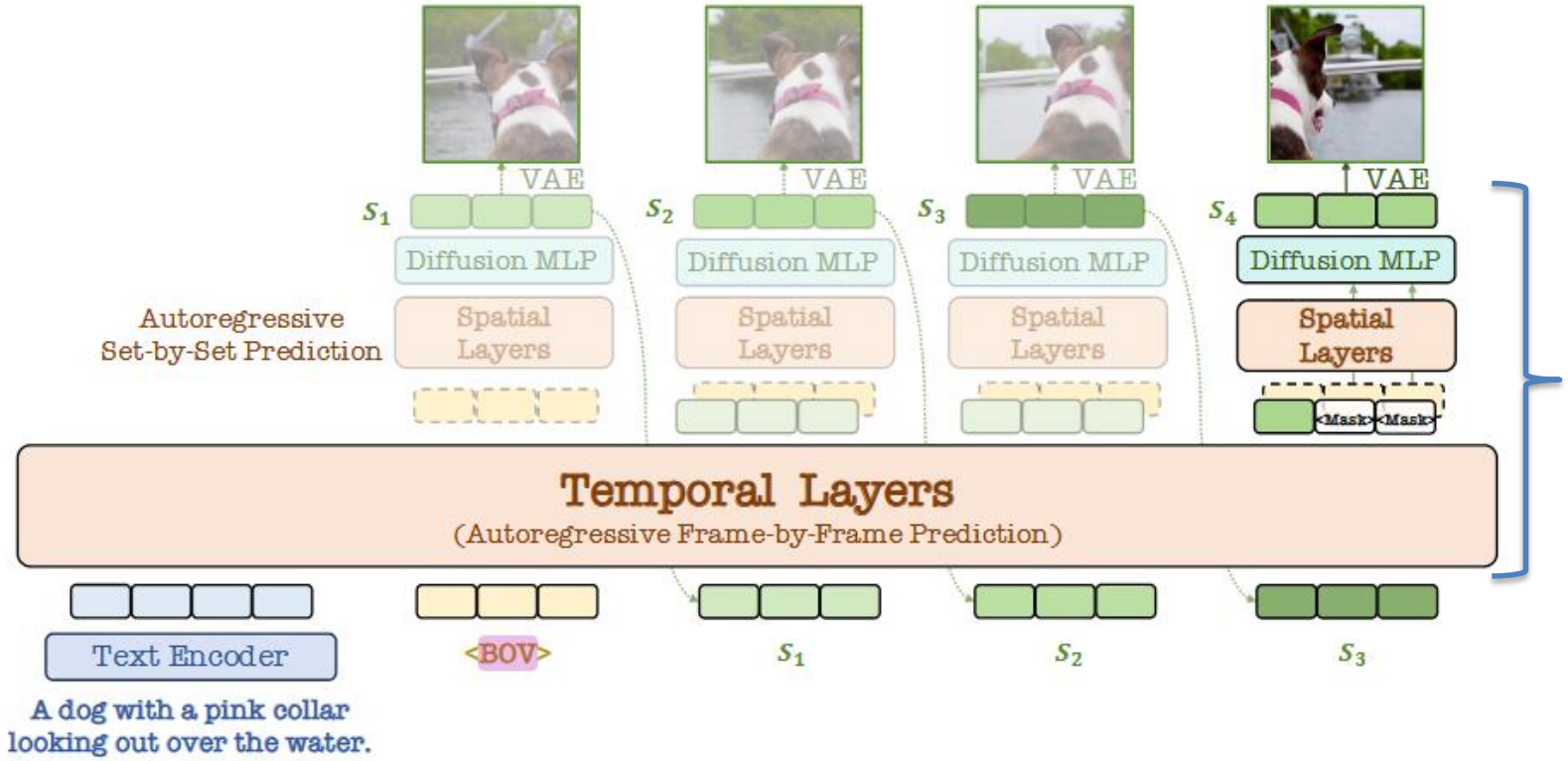
NOVA - Autoregressive Video Generation

A space shuttle launching into orbit, with flames and smoke billowing out from the engines

A 3D model of a 1800s victorian house

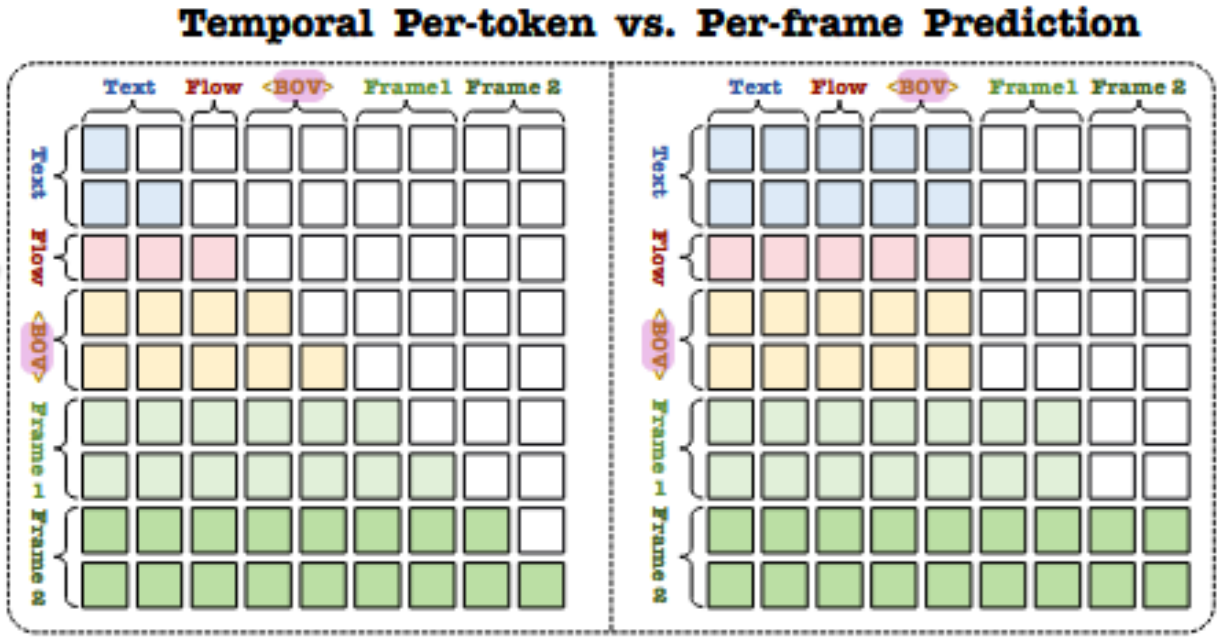
A panda drinking coffee in a cafe in Paris, tilt up

NOVA - Autoregressive Video Generation



[Deng et al. '24] Autoregressive Video Generation without Vector Quantization

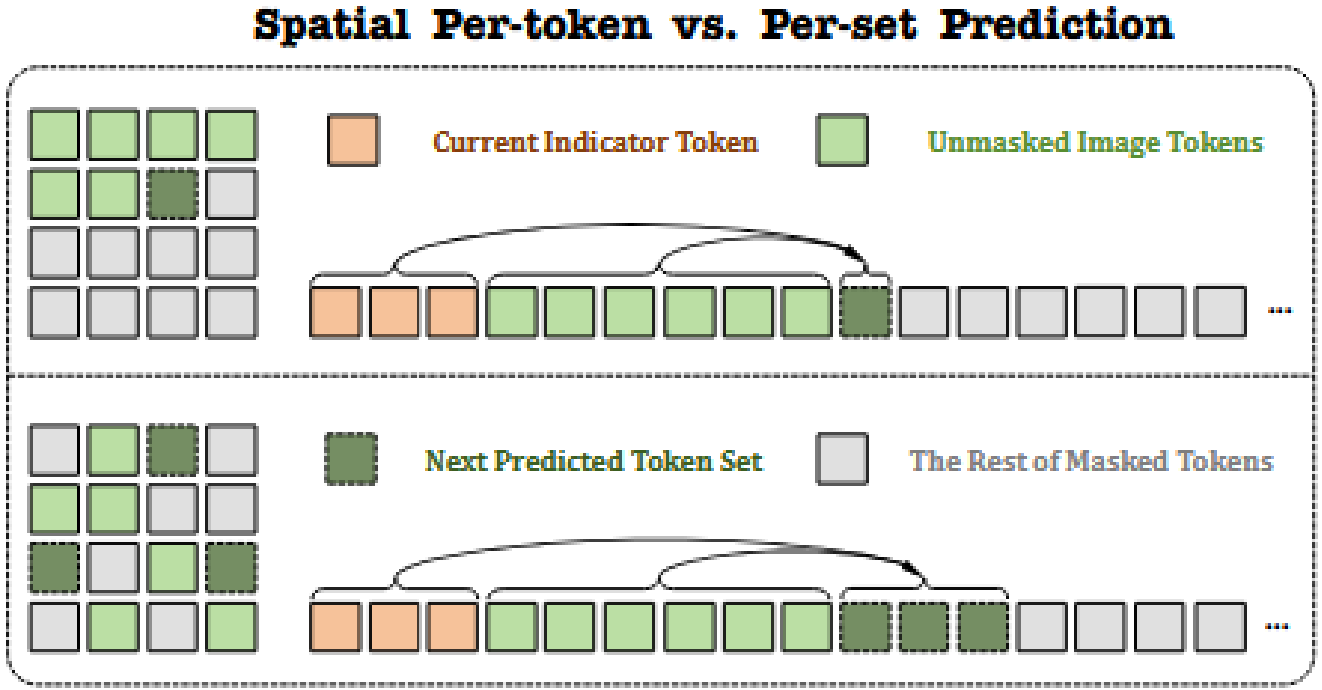
NOVA - Autoregressive Video Generation



Optical **flow** helps next frame prediction

→ Each frame attends to the text prompts, video flow, and its preceding frames.
→ All current frame tokens are visible to each other.

NOVA - Autoregressive Video Generation

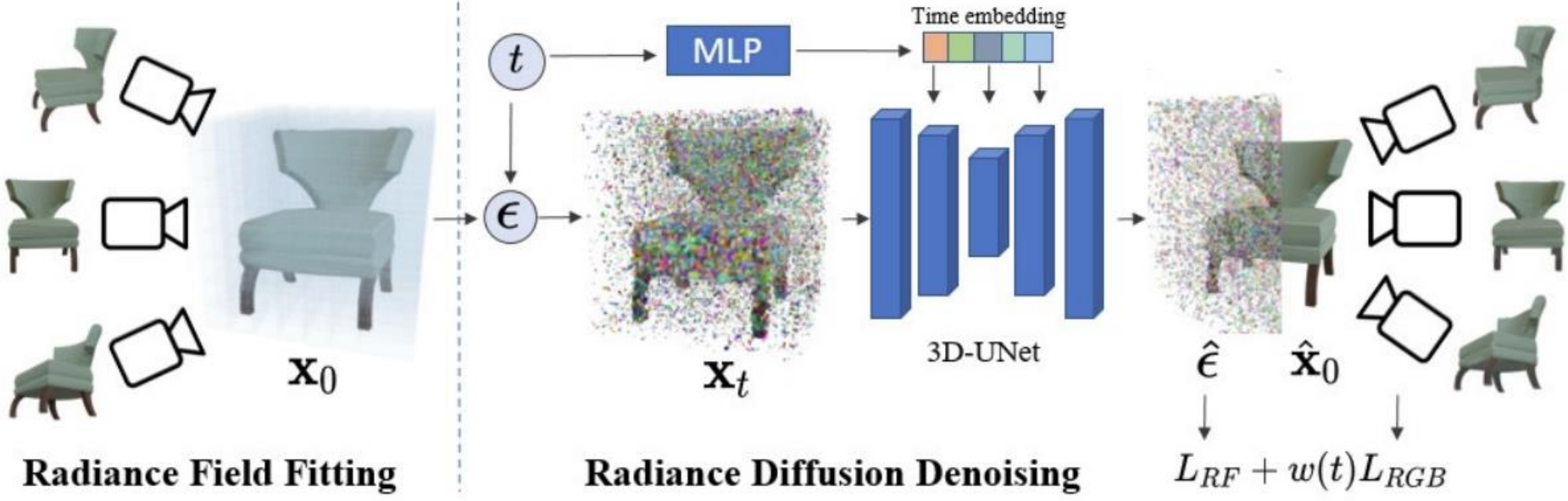


→ More efficient
→ Less cumulative error

[Deng et al. '24] Autoregressive Video Generation without Vector Quantization

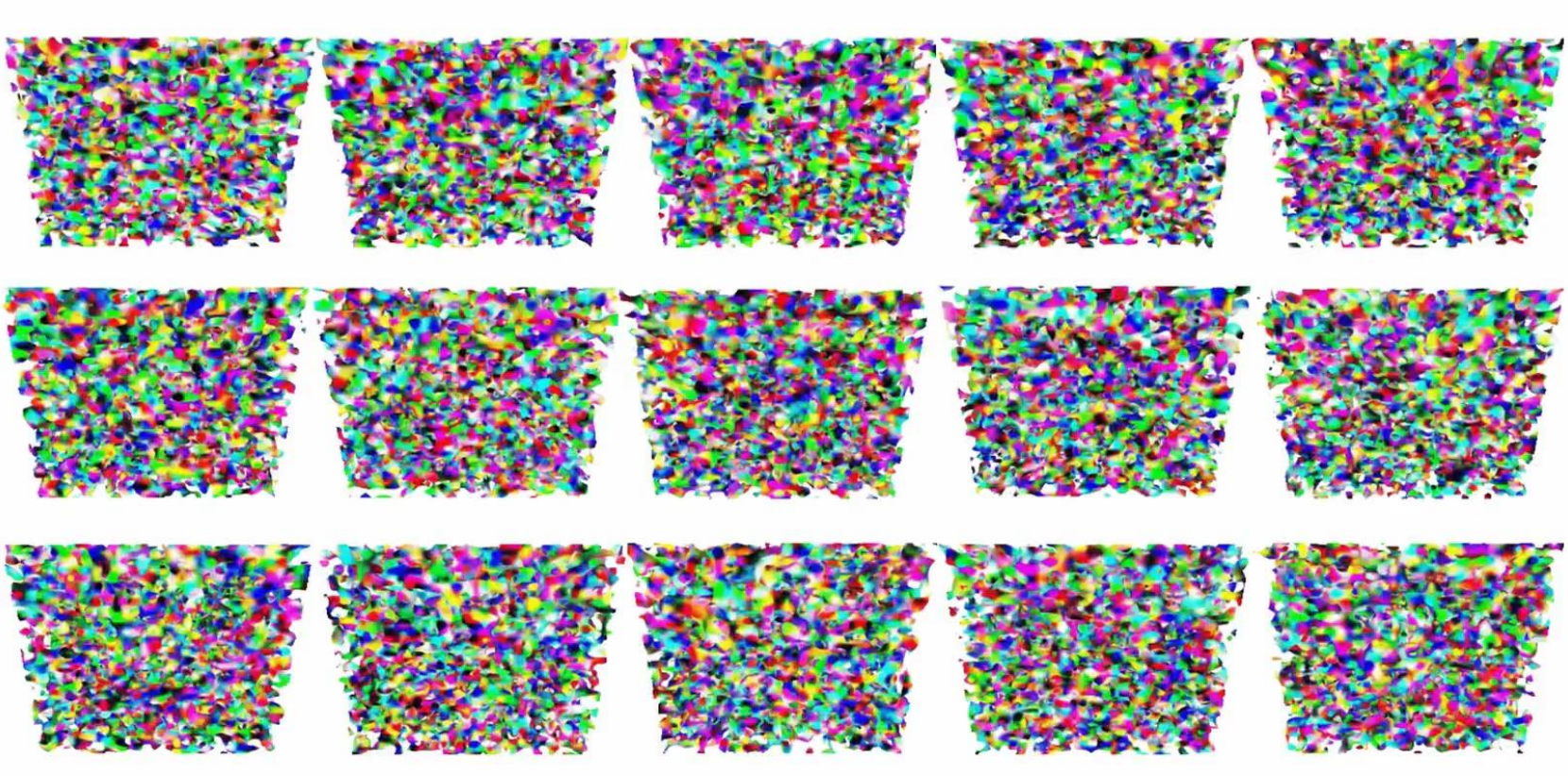
3D Aware Diffusion

DiffRF: Train with 3D Ground Truth



[Mueller et al. 23] DiffRF

DiffRF: Results

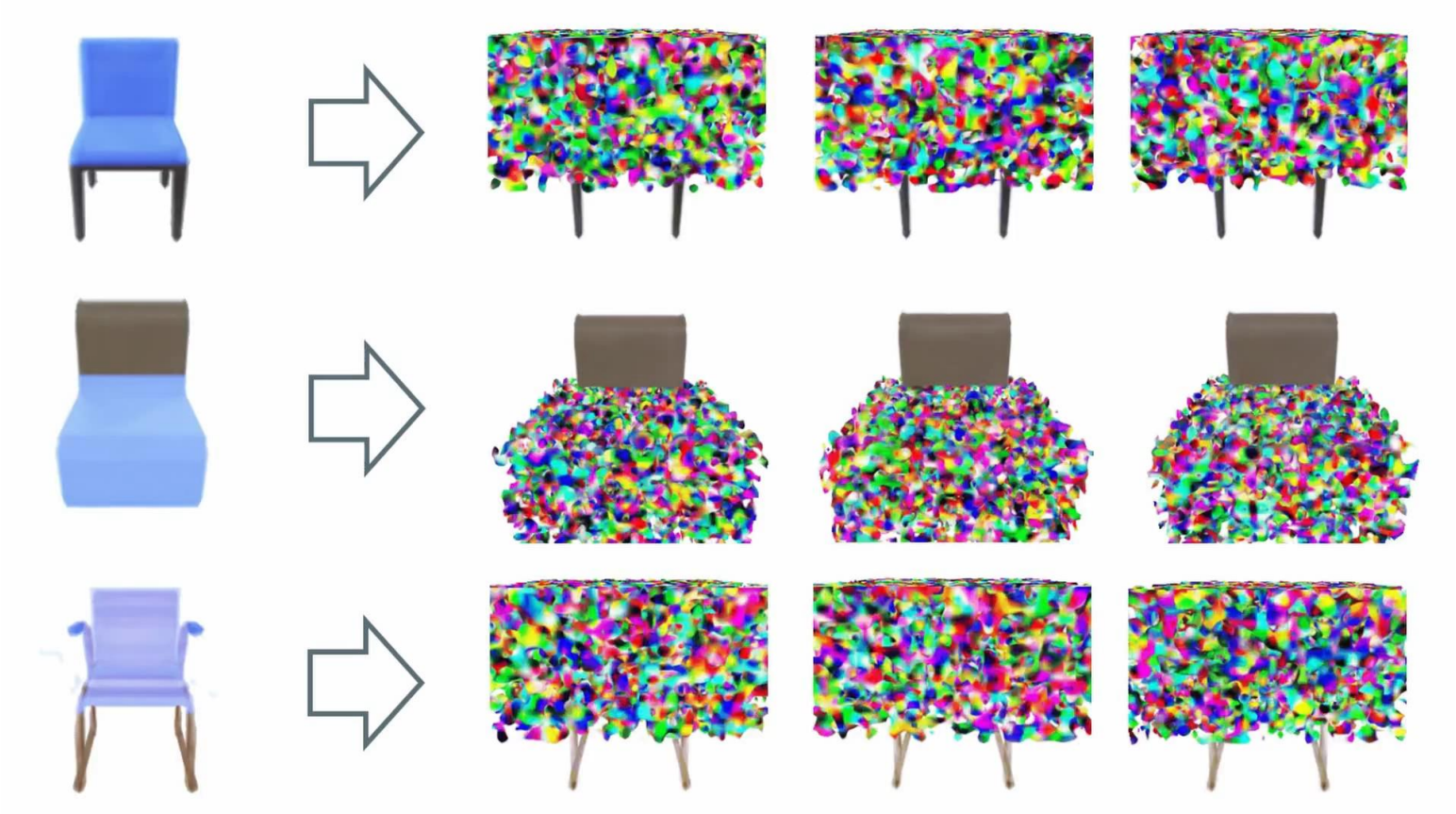


[Mueller et al. 23] DiffRF

DiffRF: Masked Predictions

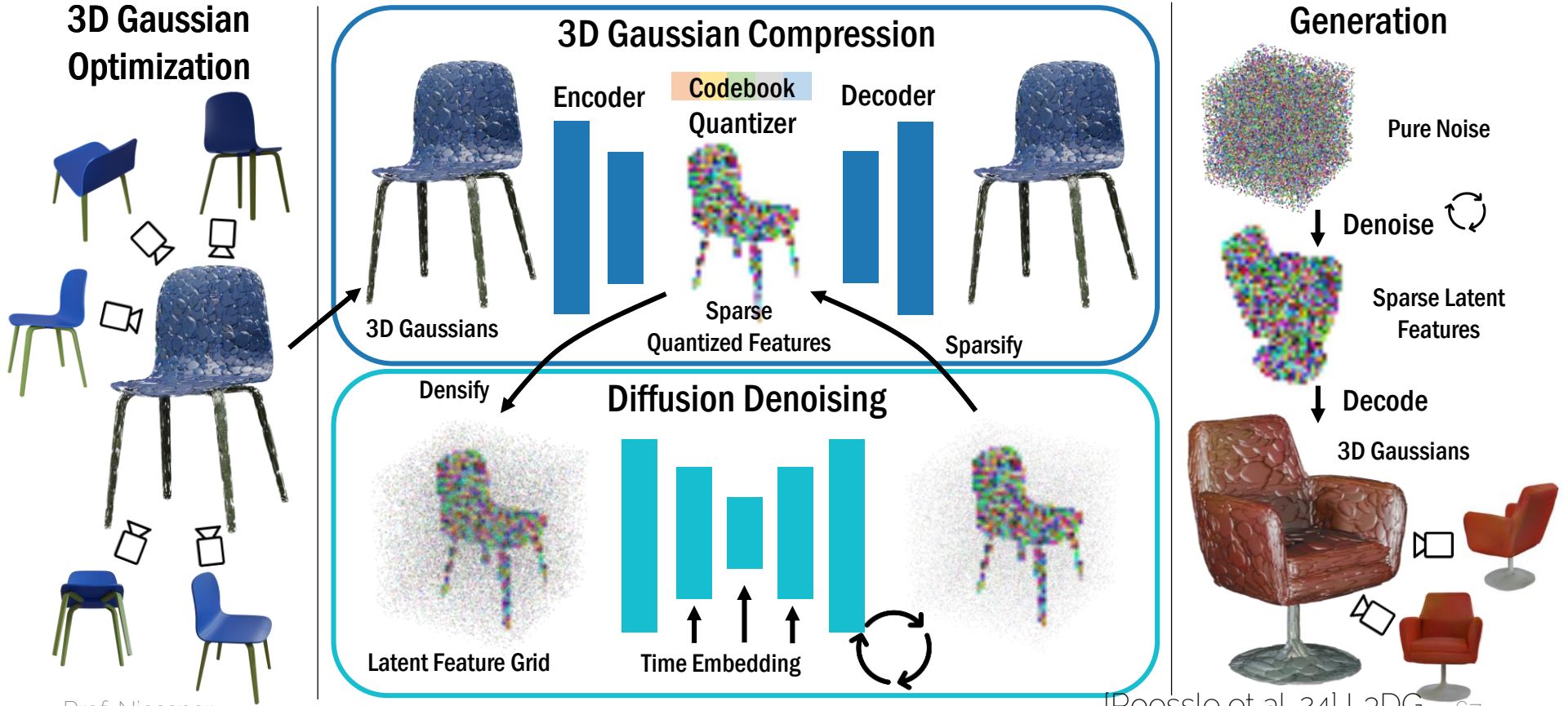
[Mueller et al. 23] DiffRF

DiffRF: Masked Predictions



[Mueller et al. 23] DiffRF

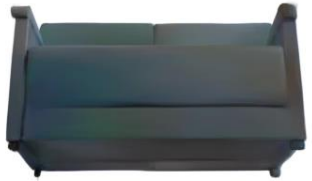
L3DG: Latent 3D Gaussian Diffusion



[Roessle et al. 24] L3DG

L3DG: Latent 3D Gaussian Diffusion

L3DG: Latent 3D Gaussian Diffusion








[Roessle et al. 24] L3DG

L3DG: Latent 3D Gaussian Diffusion



→ Generating 3D Gaussians in latent space enables higher detail on object-level generation and scalability to room-sized scenes.

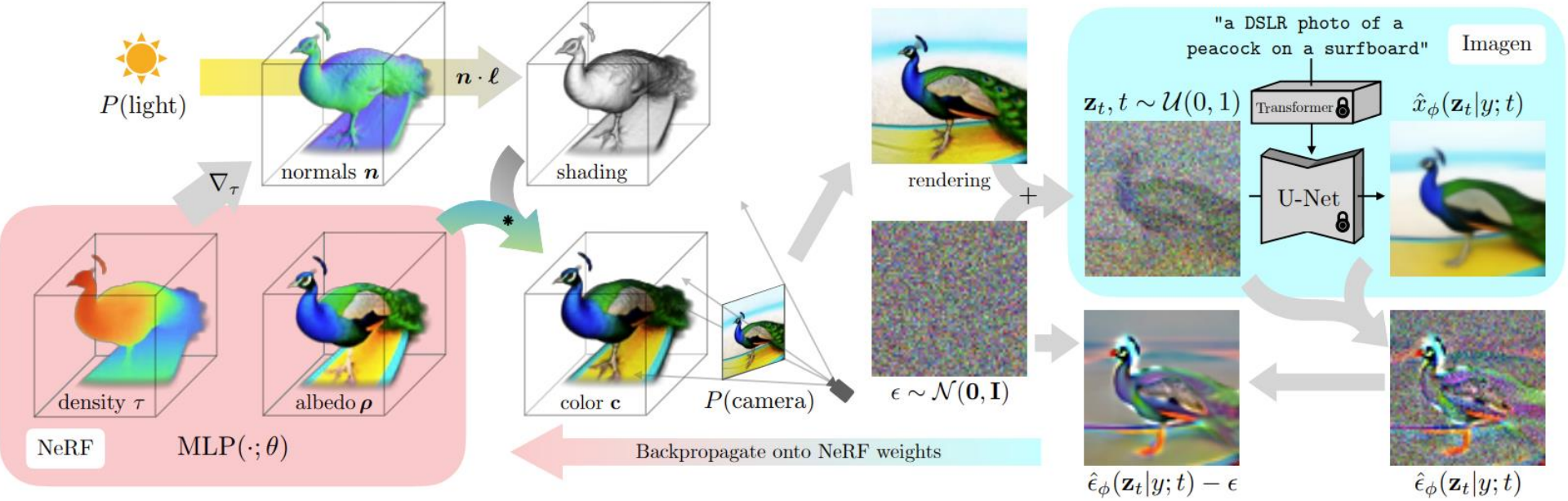
[Roessle et al. 24] L3DG

Discussion: Diffusion vs GANs

Problem is always training data…

- Diffusion: input vs output need same dimensionality
- GANs: partial information feasible (e.g., reprojection, similar to GRAF, PiGAN, EG3D)

DreamFusion



Score Distillation Sampling (SDS)

[Poole et al. 22] DreamFusion

Score Distillation Sampling (SDS)

Loss functions for diffusion models

$$\mathcal{L}_{\text{Diff}}(\phi, \mathbf{x}) = \mathbb{E}_{t\sim\mathcal{U}(0,1),\epsilon\sim\mathcal{N}(\mathbf{0},\mathbf{I})}\left[w(t)\|\epsilon_\phi(\alpha_t\mathbf{x} + \sigma_t\epsilon; t) - \epsilon\|_2^2\right]$$

Training a diffusion model:

$$\phi^* = \arg\min_{\phi} \mathcal{L}_{\text{Diff}}(\phi, \mathbf{x})$$

Sampling from a diffusion model?

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} \mathcal{L}_{\text{Diff}}(\phi, \mathbf{x})$$

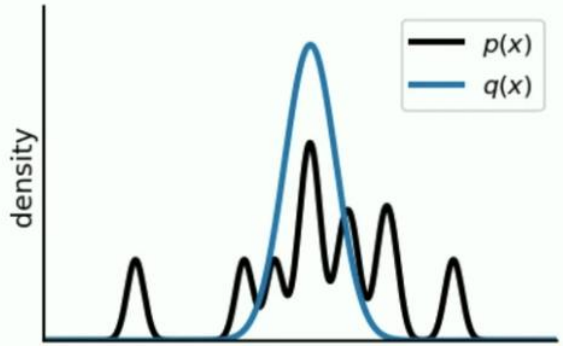
$$\nabla_\theta \mathcal{L}_{\text{Diff}}(\phi, \mathbf{x} = g(\theta)) = \mathbb{E}_{t,\epsilon}\left[w(t)\underbrace{(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon)}_{\text{Noise Residual}}\underbrace{\frac{\partial\hat{\epsilon}_\phi(\mathbf{z}_t; y, t)}{\mathbf{z}_t}}_{\text{U-Net Jacobian}}\underbrace{\frac{\partial\mathbf{x}}{\partial\theta}}_{\text{Generator Jacobian}}\right]$$

Score Distillation Sampling (SDS)

Score distillation sampling

$$\nabla_\theta \mathcal{L}_{\text{SDS}}(\phi, \mathbf{x} = g(\theta)) \triangleq \mathbb{E}_{t,\epsilon}\left[w(t)\left(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon\right)\frac{\partial \mathbf{x}}{\partial \theta}\right]$$

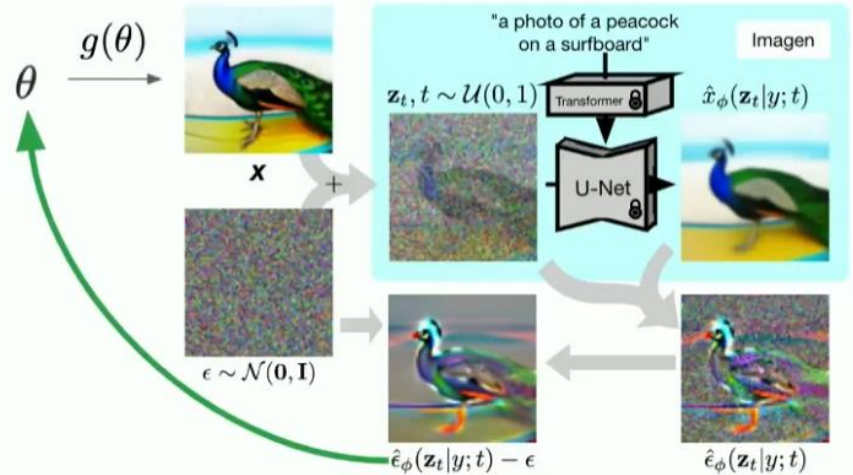
$$\mathcal{L}_{\text{SDS}}(\theta) = \mathbb{E}_t\left[w(t)\text{KL}(q(z_t; \theta, y, t)\|p_\phi(z_t; y, t))\right]$$



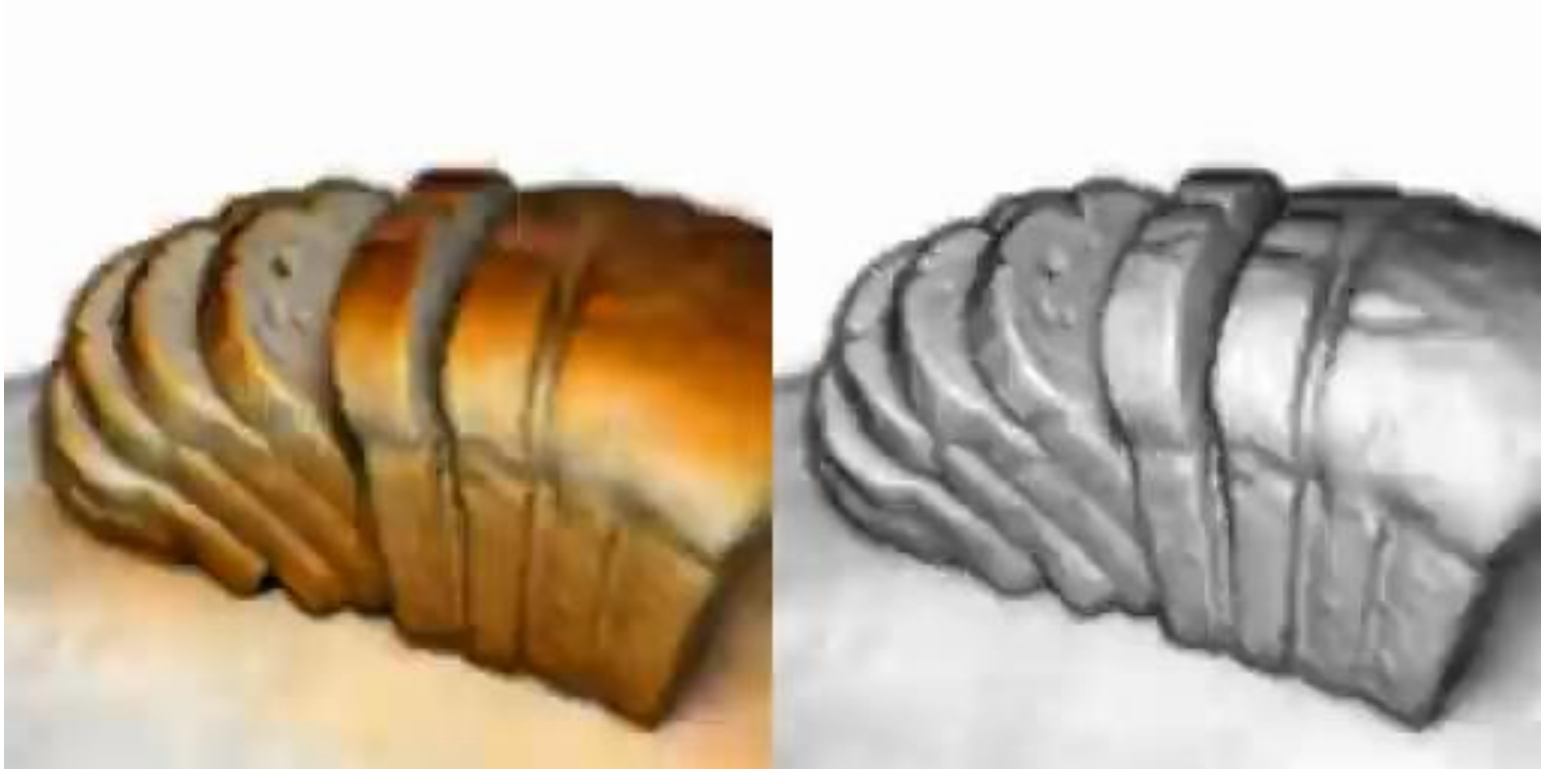
Score Distillation Sampling (SDS)

Using the score distillation loss

$$\nabla_\theta \mathcal{L}_{\text{SDS}}(\phi, \mathbf{x} = g(\theta)) \triangleq \mathbb{E}_{t,\epsilon}\left[w(t)\left(\hat{\epsilon}_\phi(\mathbf{z}_t; y, t) - \epsilon\right) \frac{\partial \mathbf{x}}{\partial \theta} \right]$$



DreamFusion



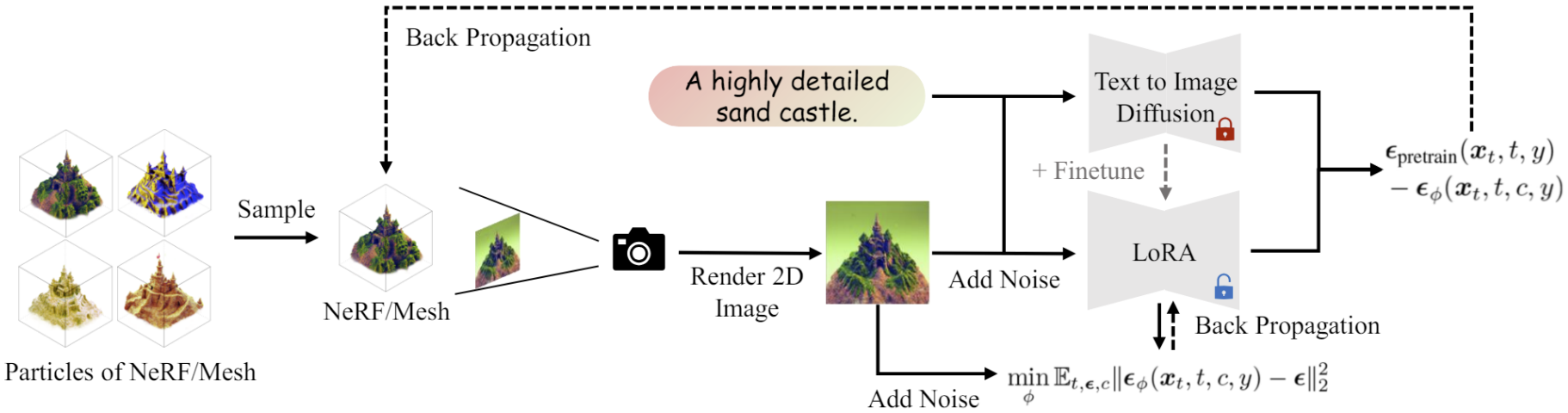
[Poole et al. 22] DreamFusion

DreamFusion

[Poole et al. 22] DreamFusion

SDS Follow Ups

- ProlificDreamer (variational SDS)



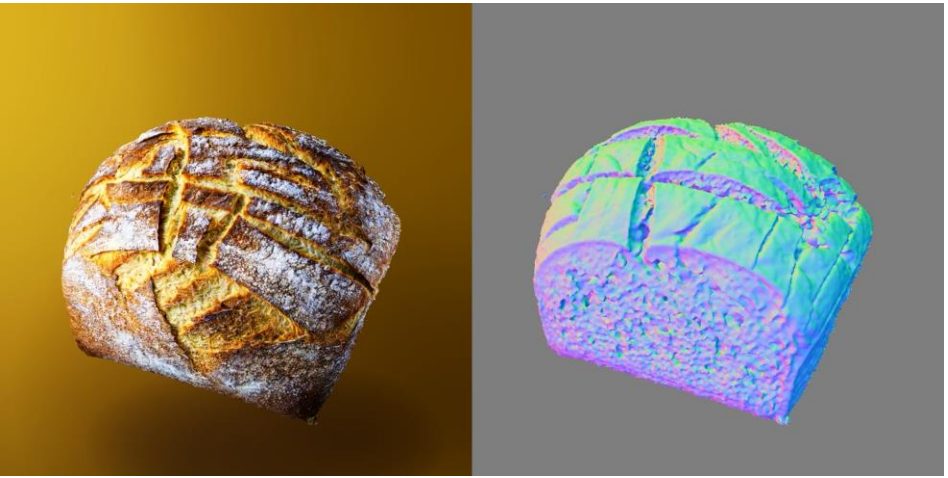
$$\nabla_\theta \mathcal{L}_{\text{VSD}}(\theta) \triangleq \mathbb{E}_{t,\boldsymbol{\epsilon},c}\left[\omega(t)\left(\boldsymbol{\epsilon}_{\text{pretrain}}(\boldsymbol{x}_t, t, y^c) - \boldsymbol{\epsilon}_\phi(\boldsymbol{x}_t, t, c, y)\right)\frac{\partial \boldsymbol{g}(\theta, c)}{\partial \theta}\right],$$

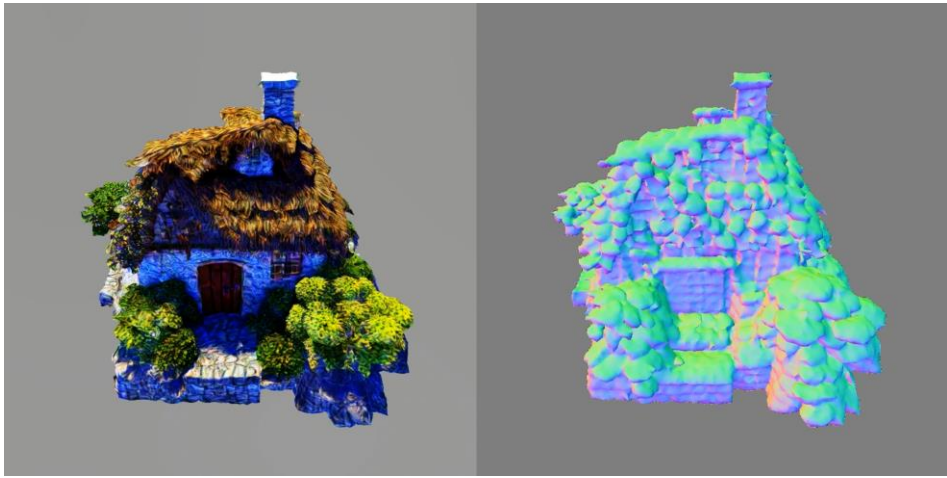
where $\boldsymbol{x}_t = \alpha_t \boldsymbol{g}(\theta, c) + \sigma_t \boldsymbol{\epsilon}$.

[Wang et al. 23] ProlificDreamer

SDS Follow Ups

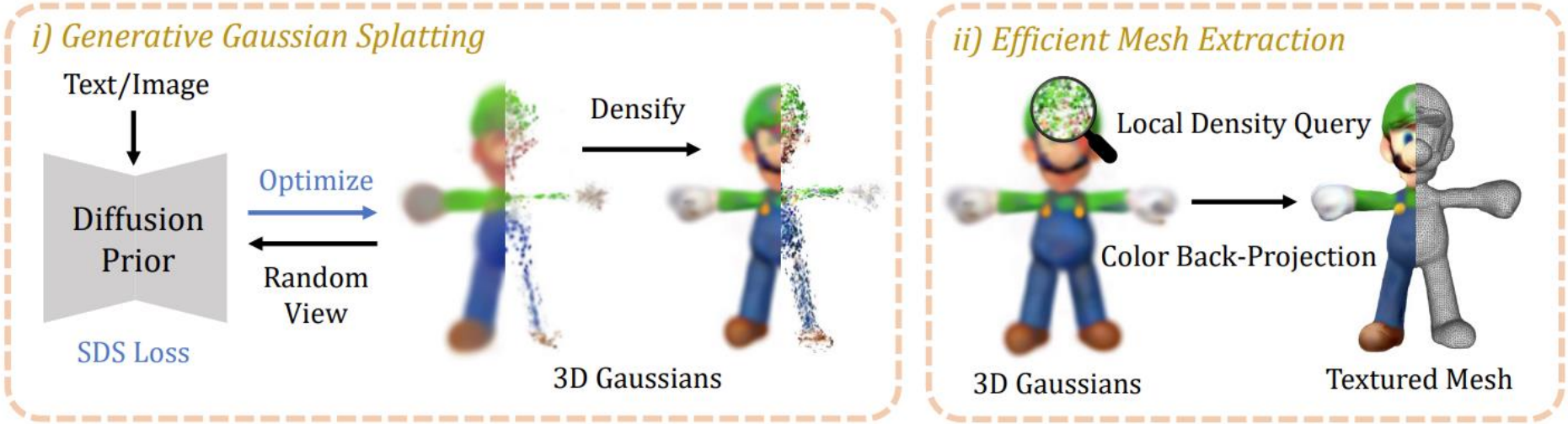
- ProlificDreamer (variational SDS)





[Wang et al. 23] ProlificDreamer

DreamGaussian

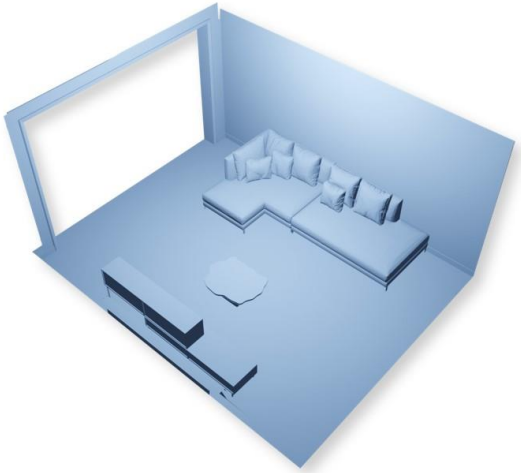


[Tang et al.. 24] Dream Gaussian

DreamGaussian

[Tang et al.. 24] Dream Gaussian

SceneTex



Scene geometry

“A Bohemian style living room”

“A country style living room”

Scene with generated texture

[Chen et al., 24] SceneTex


Baroque


Bohemian


French


Japanese

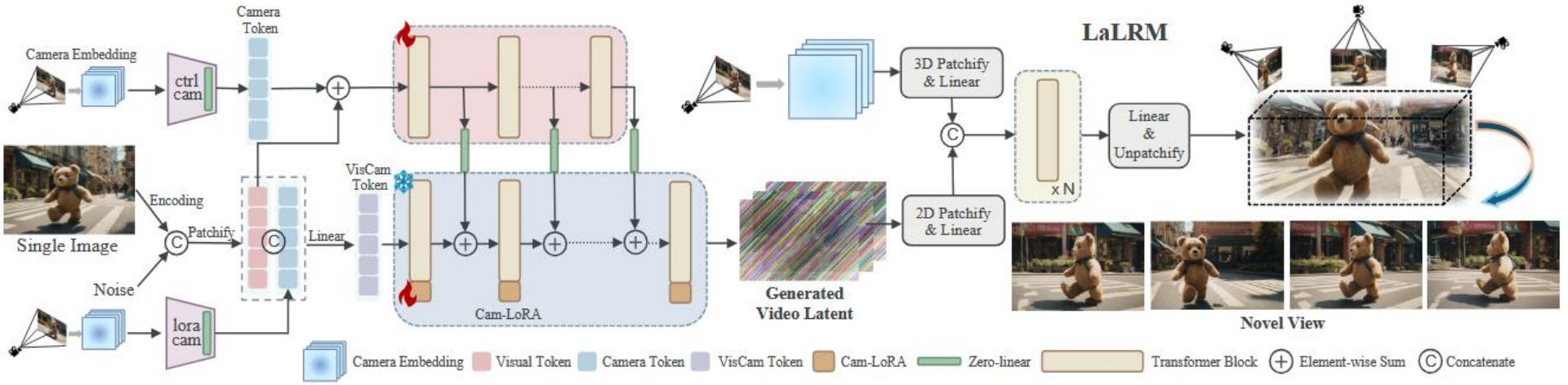
World Building

Wonderland: Navigating 3D Scenes from a Single Image

[Liang et al.. 24] Wonderland: Navigating 3D Scenes from a Single Image

Wonderland: Navigating 3D Scenes from a Single Image

Dual-branch camera conditioning

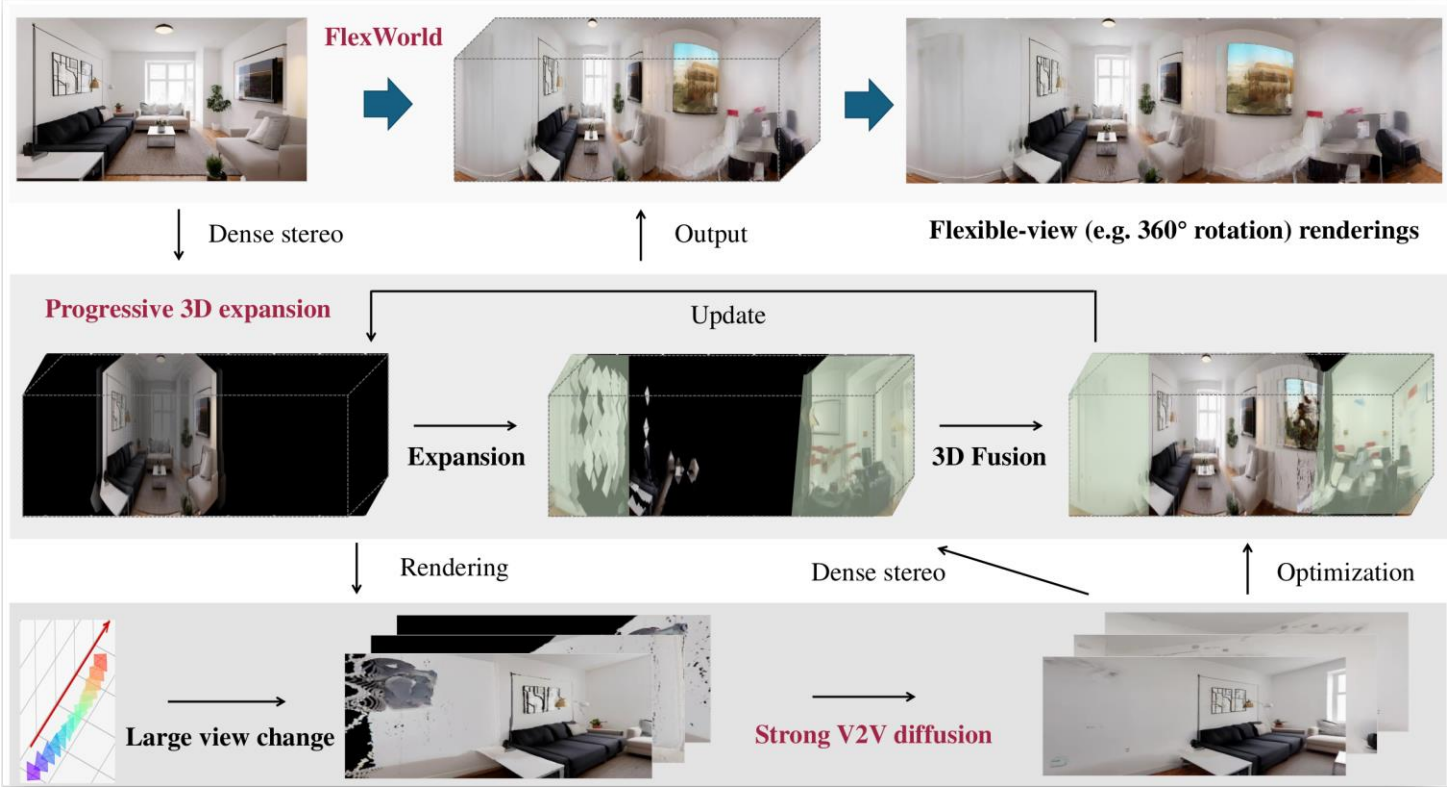


Camera-guided video diffusion model

Feed-forward reconstruction with latent large reconstruction model (LaLRM)

FlexWorld: Progressively Expanding 3D Scenes for Flexiable-View Synthesis

FlexWorld: Progressively Expanding 3D Scenes for Flexiable-View Synthesis



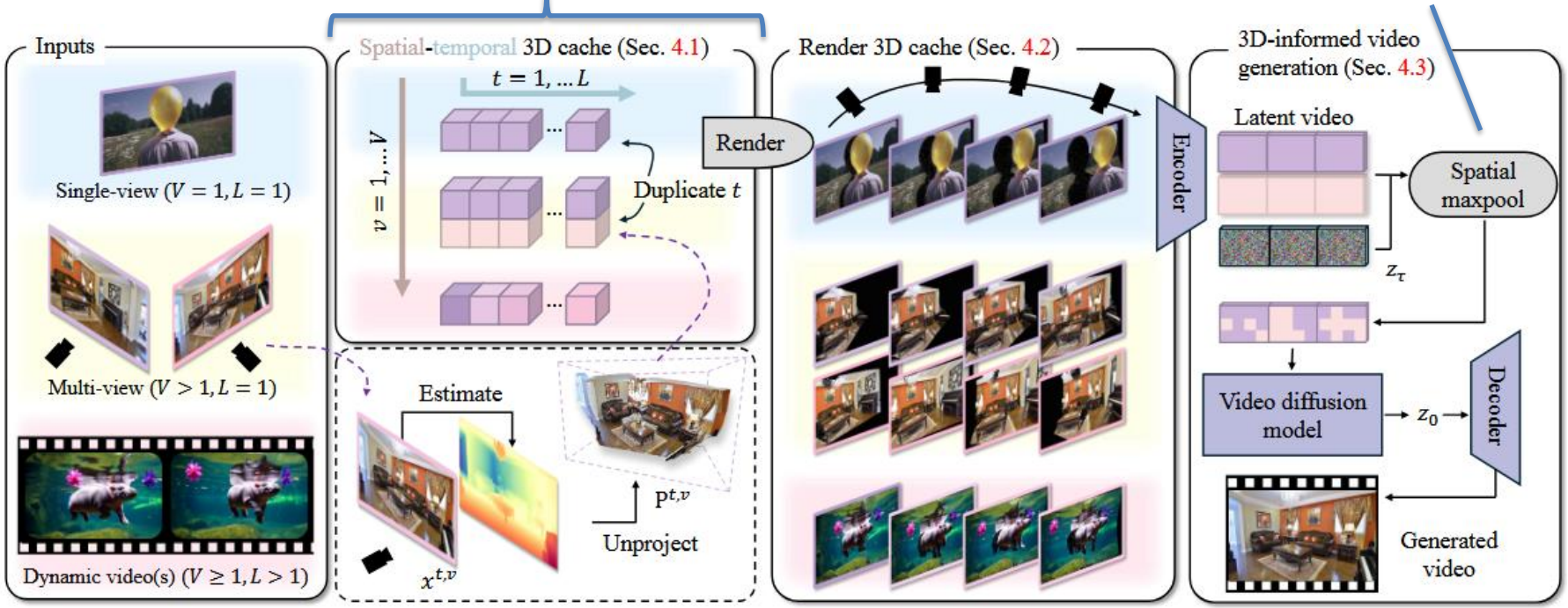
GEN3C: 3D-Informed World-Consistent Video Generation with Precise Camera Control

GEN3C: 3D-Informed World-Consistent Video Generation with Precise Camera Control

Spatial-temporal cache: one colored 3D point cloud per input view

Fuse renderings from different point clouds, by max pooling in latent space

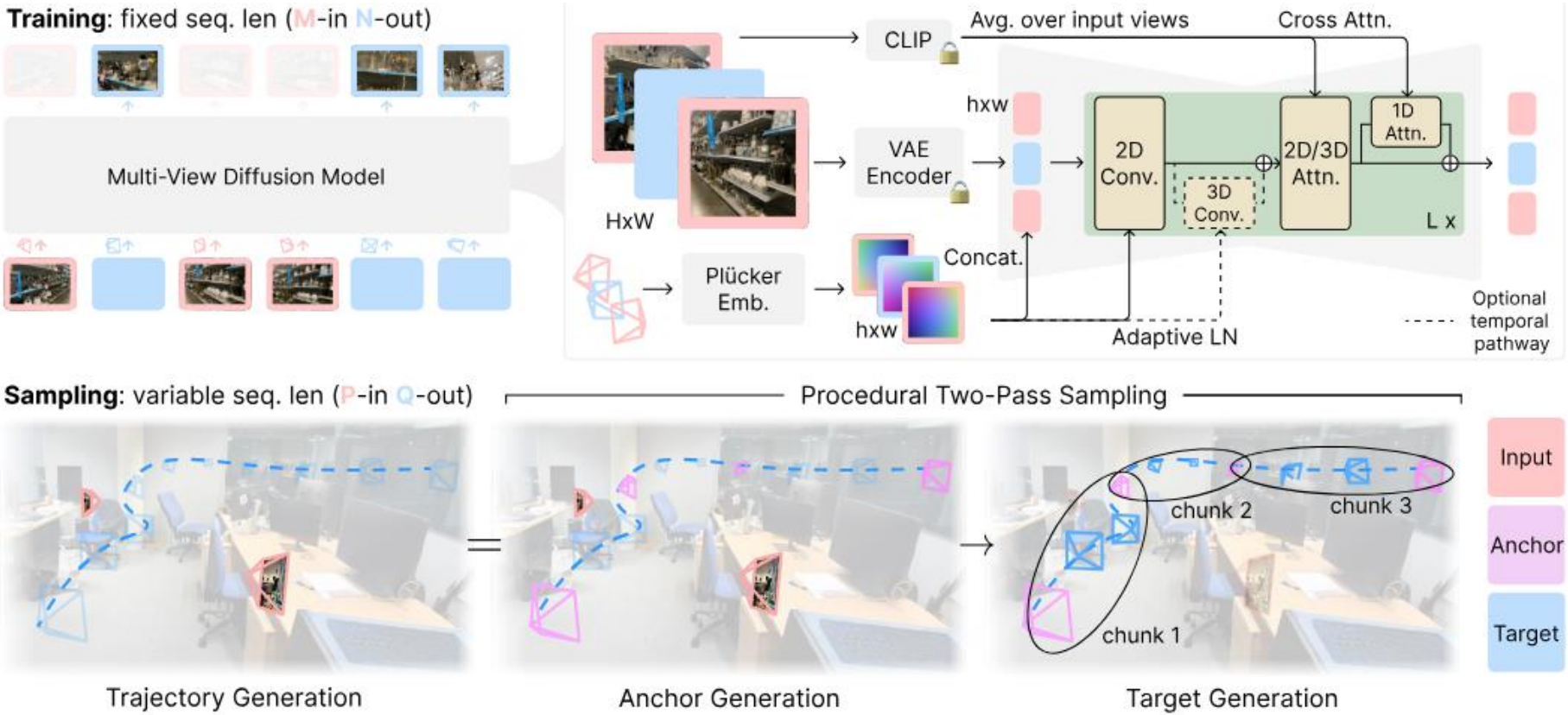
Multiple tasks supported



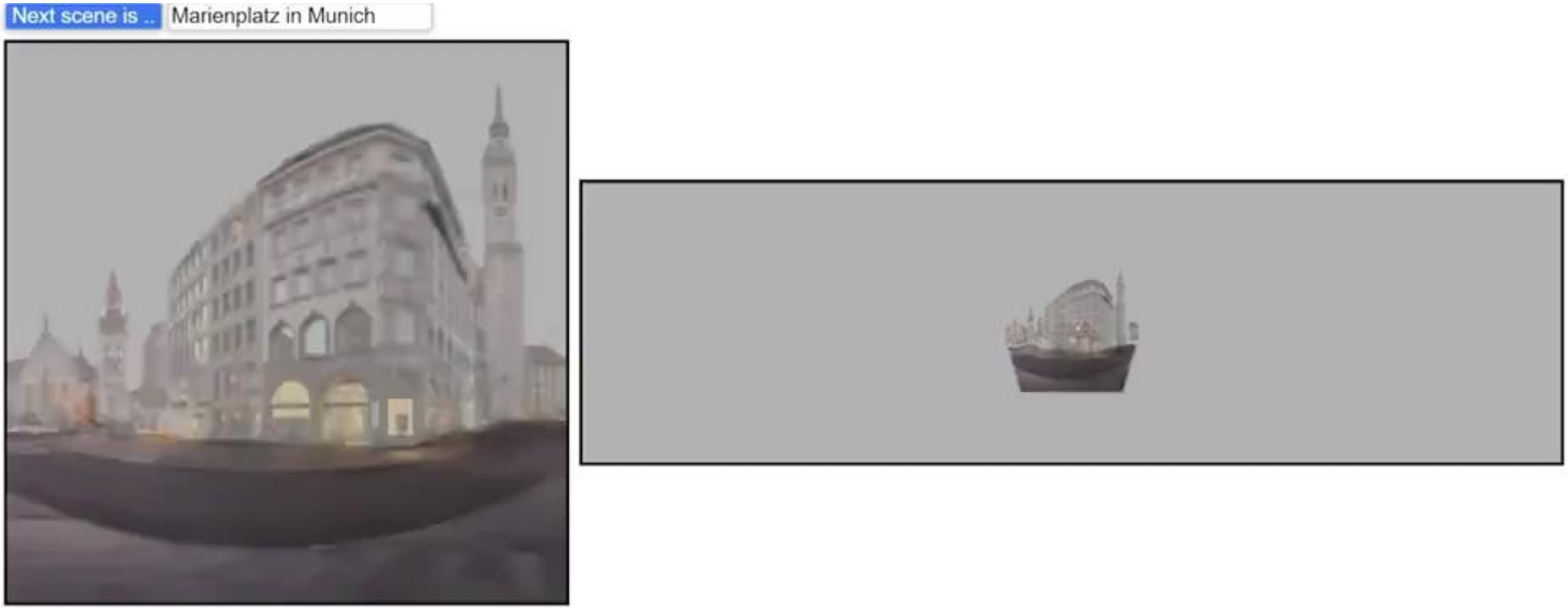
Stable Virtual Camera: Generative View Synthesis with Diffusion Models

Input: single image

Stable Virtual Camera: Generative View Synthesis with Diffusion Models

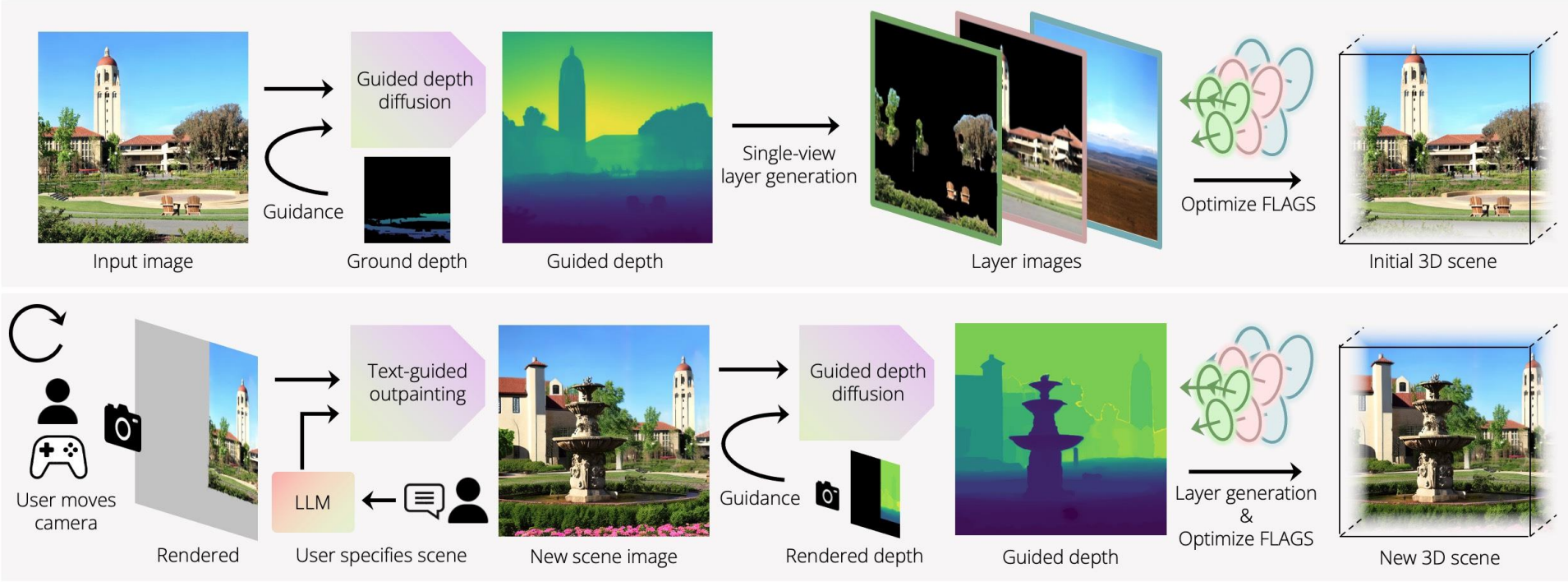


WonderWorld: Interactive 3D Scene Generation from a Single Image



[Yu et al., 25] WonderWorld: Interactive 3D Scene Generation from a Single Image

WonderWorld: Interactive 3D Scene Generation from a Single Image

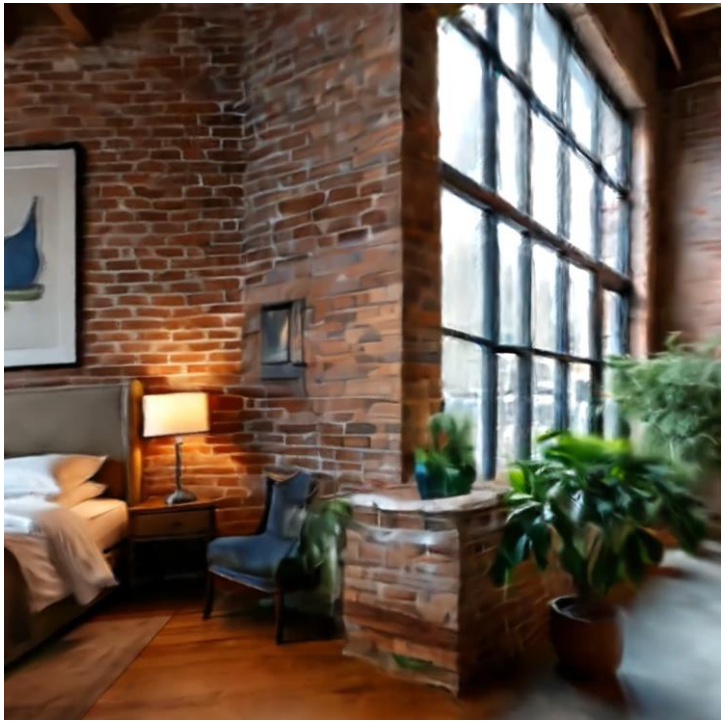


FLAGS = Fast LAyered Gaussian Surfels

[Yu et al., 25] WonderWorld: Interactive 3D Scene Generation from a Single Image

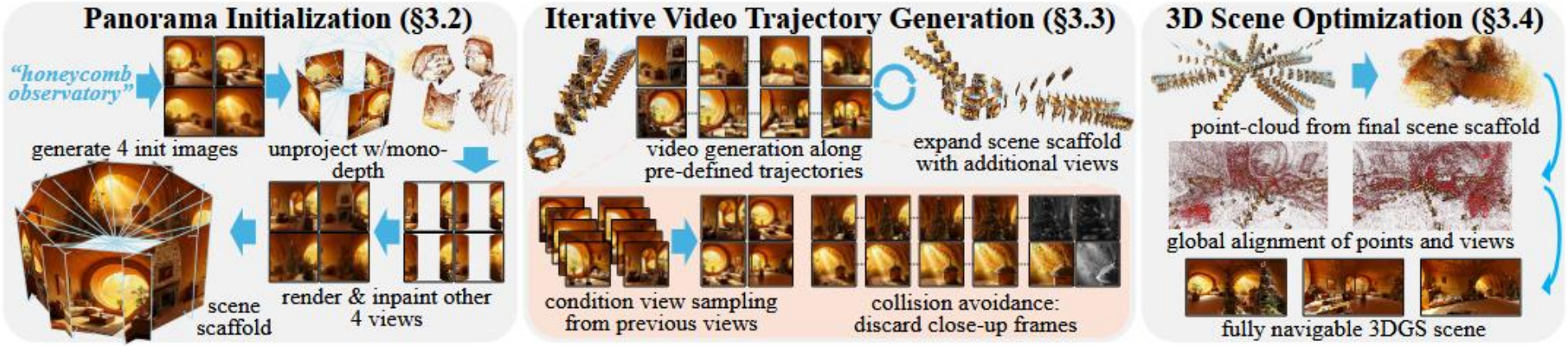
WorldExplorer: Towards Generating Fully Navigable 3D Scenes

apartment of a bioluminescent, gravity-defying, telepathic cosmic jellyfish hive



apartment of an urban industrial loft with exposed brick and ductwork

WorldExplorer: Towards Generating Fully Navigable 3D Scenes



Administratives

Current "DL Curriculum"

- MA Semester 1: I2DL (+ various intro lectures)
- MA Semester 2: ADL4CV
- MA Semester 3: Practical and/or Guided Research
- MA Semester 4: Master Thesis

Clearly should be Bachelor…

Future Project Questions

- Check out our research & see what interests you https://niessnerlab.org/publications.html
- Check out practical courses (e.g., DLinVC, etc.) https://niessnerlab.org/teaching.html
- Directly apply for GR/IDP/MA topics: https://application.vc.in.tum.de/master-application
- Reach out to PhD students / me!

Some Literature

Reading Homework

- Denoising Diffusion Probabilistic Models.
  - https://arxiv.org/abs/2006.11239
- Classifier Guided Diffusion. Diffusion Models Beat GANs on Image Synthesis
  - https://arxiv.org/abs/2105.05233
- Classifier-Free Guidance. GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models
  - https://arxiv.org/abs/2112.10741
- CLIP Guidance. Hierarchical Text-Conditional Image Generation with CLIP Latents
  - https://arxiv.org/abs/2204.06125

Literature

- CVPR 2022 Tutorial on Denoising Diffusion-based Generative Modeling
  - https://cvpr2022-tutorial-diffusion-models.github.io/
- Tackling the Generative Learning Trilemma with Denoising Diffusion GANs
  - https://arxiv.org/abs/2112.07804
- Deep Unsupervised Learning using Nonequilibrium Thermodynamics
  - https://arxiv.org/abs/1503.03585
- Denoising Diffusion Probabilistic Models
  - https://arxiv.org/abs/2006.11239
- Diffusion Models Beat GANs on Image Synthesis
  - https://arxiv.org/abs/2105.05233

Thanks for watching!